

Дискретный анализ

Вайб-лекция 2: Поиск

Андрей Калинин

Дискретный анализ, 2026/27

19 сентября 2026

Будьте внимательны: возможны ошибки и галлюцинации!

Вопрос с прошлой лекции

Почему `std::sort` на *обратном* входе быстрее,
чем на *отсортированном*?

- Вопрос, заданный на прошлой лекции.
- Ответ измерен: копия `introsort` со счётчиками, наложенная на библиотеку, $n = 10^6$.
- Три числа и одна причина.

вопрос — прошлая лекция; ответ — `../20260912/demo/shapes/`, вывод `out-shapes.txt`.

Ответ: дерево разбиений и одна перестановка

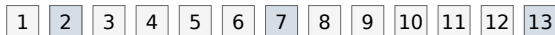
<code>uint32, n = 10⁶</code>	обратный	прямой	
сравнений на ключ	18,13	25,60	–29 %
разбиений	65 535	98 302	в 1,5 раза меньше
вырожденных разбиений	0	32 767	ровно треть
обменов	565 534	131 069	в 4,3 раза <i>больше</i>
время, мс	4,56	6,35	

`__unguarded_partition_pivot` берёт медиану из трёх и *переставляет её в начало* массива: на отсортированном входе этот обмен портит порядок, и ровно треть разбиений вырождена. Обменов в 4,3 раза больше, а время меньше — оно пошло за числом сравнений. Копия `introsort` сходится с библиотекой до 14 сравнений на миллион ключей.

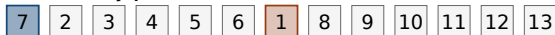
Откуда берутся вырожденные разбиения

Первое разбиение честно пополам. Вырождается **следующее**.

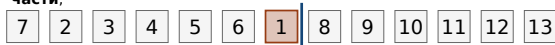
1. вход отсортирован; median-of-three смотрит на $\text{first}+1$, mid , $\text{last}-1$ (подсвечены). Медиана — 7;



2. `__move_median_to_first` меняет медиану с первым элементом: опорный 7 встаёт в начало, а **минимум 1 вытеснен внутрь**;



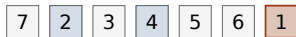
3. разбиение отрабатывает как надо: 7/6, **ровно пополам**. Но вытесненный минимум остался **в конце левой части**;



левая, 7

правая, 6

4. рекурсия входит в левую часть. Её $\text{last}-1$ — это тот самый минимум, он и становится третьим кандидатом:



$\text{median}(2, 4, 1) = 2$ — второй по малости элемент диапазона. Разбиение 2/5: вырожденное.

Минимум вытесняется ровно в $\text{last}-1$ — туда, где следующий уровень ищет третьего кандидата. На **обратном** входе вытесняется **максимум**: он попадает в first правой части, а first median-of-three **не читает** никогда.

Замер, $n = 10^6$: отсортированный — 32 767 вырожденных из 98 302 (треть); случайный — 6 107 из 104 010 (5,9%); обратный — 0 из 65 535.

`stl_algo.h`: `__unguarded_partition_pivot`, стр. 1893–1899, и `__move_median_to_first`, стр. 85–103 (g++ 13.3); прогон —

Задача дня — и пять минут на бумаге

Массив отсортирован. Найти **первый** элемент, который $\geq x$.

- Это `lower_bound`: позиция, куда вставился бы x .
- Все его знают.
- Напишите на листе бумаги. Любой язык, можно псевдокод. Пять минут.
- Листы остаются у вас — это не контрольная.

Разбор: от инварианта к циклу

```
1 // Инвариант: ответ, если он есть, лежит в [lo, hi).
2 size_t lo = 0, hi = n;           // hi — за конец
3 while (lo < hi) {
4     size_t mid = lo + (hi - lo) / 2; // почему не (lo + hi) / 2 — позже
5     if (a[mid] < x) lo = mid + 1;    // a[mid] заведомо не ответ
6     else hi = mid;                  // mid сам мог быть ответом
7 }
8 return lo;                         // первый >= x; n, если таких нет
```

- $hi = mid$, а не $mid - 1$: ищем *первый* $\geq x$.
- Длина отрезка убывает строго — цикл конечен. Отказ: $lo = mid$, и вечный цикл на отрезке длины 1.
- `lower_bound` и `upper_bound` — один цикл с другим сравнением: $a[mid] < x$ против $a[mid] \leq x$.

свой код, не цитированный (правило рецепта); тот же цикл в репозитории — `naive` в `demo/bench-binsearch/bench.cpp`, строки 67–77.

У кого получилось без ошибки?

демо

Проверьте свой лист по трём входам — и только потом поднимайте руку.

	вход	x	должно вернуть
1	{ }	42	0
2	{1, 3, 5, 7}	9	4
3	{5, 5, 5, 5, 5}	5	0 , а upper_bound — 5

- В 1 и 2 ответ — это **n**, позиция за последним элементом. Не **-1**: мы вернули, куда *вставить*, а вставить можно и в конец — поэтому `a[ответ]` читать нельзя. `std::lower_bound` возвращает ровно это, итератор `last`.
- В 1 пусто и «не найдено» дают один и тот же **0**: отдельная ветка «если массив пуст» уже ошибка.

Минута на проверку. Потом — поднимаем руки.

А на массиве в два миллиарда элементов?

демо

```
1 int lo = 1073741824, hi = 2147483647;    // 2^30 и 2^31 - 1
2 int mid = (lo + hi) / 2;                  // тот же цикл, прежняя середина
3 printf("mid = %d\n", mid);
```

mid = -536870912

- Массив на 2^{30} элементов — около 8 ГБ: таким тестом ошибку не поймать.
- Руки, поднятые на предыдущем слайде, эту проверку не проходят тоже.

живьём: demo/verify/demo.c — 7 строк; эталонный вывод — demo/verify/expected-output.txt, шаг 1.

Фольклор — теперь по источникам

- Бентли, *Programming Pearls* (Addison-Wesley, 1986), Column 4 «Writing Correct Programs», печатная с. 36: «...given ample time, only about ten percent of professional programmers were able to get this small program right.»
- Тем же абзацем, там же — фраза, которую все знают как «Кнута»: «...while the first binary search was published in 1946, the first published binary search **without bugs** did not appear until 1962.»
Это **Бентли**, а не Кнут: в 6.2.1 такой строки нет. Блох цитирует её в своей редакции — «...works correctly for all values of n ...»
- Блох, 02.06.2006. Его собственный код в JDK был `int mid = (low + high) / 2;`
«This bug can manifest itself for arrays whose length (in elements) is 2^{30} or greater (roughly a billion elements)»
«It was reported to Sun recently when it broke someone's program, after lying in wait for nine years or so»

Бентли — *Programming Pearls*, 1-е изд., с. 36, прочитано с изображения страницы. Блох — запись в блоге Google Research,

02.06.2006 (адрес в `sources.md`); все три цитаты сверены дословно.

Учебник, который учит доказывать программу, — и сам даёт эту строку

Бентли, Column 4 «Writing Correct Programs», §4.2 — его *доказанный* вариант поиска, напечатанный на одной странице дважды:

$M := (L+U) \text{ div } 2$

Строка 34 — внутри цикла, строка 42 — в том же разборе. Строкой ниже Бентли пишет, что делает `div`; про переполнение — ни слова.

Это та же ошибка, с которой начался день: при $L + U > 2^{31} - 1$ сумма знаковых целых даёт отрицательный `mid`. `div` тут ни при чём — при чём сложение.

Бентли, *Programming Pearls*, 1-е изд., печатная с. 37 (DjVu 42); фрагмент — [pages/p37-detail-midpoint.png](#).

The `break` statement terminates the enclosing `loop`. This statement computes M , the midpoint of the range:

$M := (L+U) \text{ div } 2$

The `div` operator implements integer division: $6 \text{ div } 2$ is 3, as is $7 \text{ div } 2$. The program is now

$L := 1; U := N$
`loop`

печатная с. 37, §4.2 — курсивом там же:
«This statement computes M , the midpoint of the range»

Откуда мы знаем (1): чем ловят то, что не ловится тестами

демо

```
1 #include <stdio.h>
2 int mid_buggy(int lo, int hi) { return (lo + hi) / 2; }
3 int main(void) {
4     int lo = 1 << 30, hi = 2147483647;
5     printf("lo=%d hi=%d -> mid=%d\n", lo, hi, mid_buggy(lo, hi));
6     return 0;
7 }
```

```
1 $ gcc -O0 -fsanitize=signed-integer-overflow -o mid-demo demo.c
2 $ ./mid-demo
3 demo.c:2:44: runtime error: signed integer overflow:
4     1073741824 + 2147483647 cannot be represented in type 'int'
5 lo=1073741824 hi=2147483647 -> mid=-536870912
6
```

demo/verify/demo.c, строки 1-7; вывод — demo/verify/expected-output.txt, шаг 1. gcc 13.3.0,

-fsanitize=signed-integer-overflow, стенд M1.

Откуда мы знаем (2): доказано для всех входов — без массива

демо

CBMC — C Bounded Model Checker из набора **CPROVER** (cprover.org).
__CPROVER_assume сужает перебор, __CPROVER_assert задаёт доказываемое; это не функции, а директивы самого CBMC.

```
2 int mid_buggy(int lo, int hi) { return (lo + hi) / 2; }
3 int mid_safe (int lo, int hi) { return lo + (hi - lo) / 2; }
4
5 void check_buggy(void) {
6     int lo, hi;
7     __CPROVER_assume(0 <= lo && lo <= hi && hi < 2147483647);
8     int m = mid_buggy(lo, hi);
9     __CPROVER_assert(lo <= m && m <= hi, "midpoint stays inside [lo, hi]");
10 }
```

check_buggy — FAILED, check_safe — SUCCESSFUL. CBMC переводит программу в формулу и отдаёт SAT-решателю: не «много тестов», а **все** пары $lo \leq hi$, без массива. И выдаёт контрпример — 456 458 241, 2 043 346 942 → −897 581 056: числа нашёл решатель, не мы.

Откуда мы знаем (3): одно выражение — две модели «целого»

демо

```
10 void check_math(void) {  
11     long long lo, hi;  
12     __CPROVER_assume(0 <= lo && lo <= hi && hi < 2147483647LL);  
13     long long m = (lo + hi) / 2;  
14     __CPROVER_assert(lo <= m && m <= hi, "midpoint inside [lo, hi]");  
15 }
```

check_machine — **FAILED**; check_math — **SUCCESSFUL**.

check_machine — это строки 4-9, дословно check_buggy с прошлого слайда под другим именем: то же $(lo + hi) / 2$, только над `int`. Изменился **не алгоритм, а тип** — одно выражение, две модели «целого», два разных ответа о его корректности.

demo/verify/bs2.c, на слайде строки 10-15, файл целиком — 1-15; вывод — expected-output.txt, шаг 3. CBMC 5.95.1.

Откуда мы знаем (4): и long long — не математика

демо

check_math прошёл **не потому**, что long long — математика, а потому, что assume держал $lo, hi < 2^{31}$: сумма не выходила за 64 бита. Снимем ограничение:

```
11 void check_wide_unbounded(void) {           // same code, range assumption dropped
12     long long lo, hi;
13     __CPROVER_assume(0 <= lo && lo <= hi);
14     long long m = (lo + hi) / 2;
15     __CPROVER_assert(lo <= m && m <= hi, "midpoint inside [lo, hi]");
16 }
```

check_wide_bounded — **SUCCESSFUL**, check_wide_unbounded — **FAILED**.
Контрпример: $lo \approx 3,96 \cdot 10^{18}$, $hi \approx 7,19 \cdot 10^{18}$ — сумма за $2^{63} - 1$, и $m \approx -3,65 \cdot 10^{18}$.

У CBMC **нет** режима математических целых: любой тип C для него — битовый вектор фиксированной ширины. Мы взяли не математику, а модель *пошире*. **Вывод сегмента:** доказательство ровно настолько хорошо, насколько хороша его модель машины.

	задача	результат
L1	бинарный поиск	0 ошибок из 11 генераций (6 моделей): все написали $\text{left} + (\text{right} - \text{left}) / 2$
L2	<code>int average(int a, int b)</code>	5 из 11 написали $(a + b) / 2$ — та же ошибка
L3	то же, с объяснением	3 из этих 5 в том же ответе объясняют переполнение
L4	<code>int my_abs(int x)</code>	3 из 11: <code>return -x;</code> без защиты от <code>INT_MIN</code>
L5	<code>Comparator<Integer></code>	0 из 11: все взяли <code>Integer.compare</code>
L6	все три ловушки	5 из 11 прошли все три чисто

Оговорка: по одной генерации на пару «модель × задача», разбор ответа *статический*. Это воспроизводимая демонстрация, а не статистика о моделях. Вывод: **ошибка Блоха защищена своей славой, а не пониманием машинной арифметики** — её близнец под другим именем проходит насквозь.

16.09.2026, по одному вызову на пару; `demo/llm-binsearch/ — summary.txt` (11 строк) и `traps/out-analyse.txt`; ответы моделей сохранены дословно.

Эксперимент с моделями целиком

```
1 avg – среднее двух int; ловушка (a + b) / 2
2   модель                первый return          вердикт    упомянул переполнение
3   MiniMax-M3             (a + b) / 2             ОШИБКА     нет
4   deepseek-v4-pro        qa + qb + (ra + rb) / 2    безопасно да
5   glm-5.2                (a + b) / 2             ОШИБКА     да
6   or-claude-sonnet-5     (a & b) + ((a ^ b) >> 1)    безопасно да
7
```

На слайде — начало таблицы avg; целиком вывод лежит в файле источника, там три таблицы (L2–L7), кто что написал и кто предупредил и всё равно ошибся.

Побочный результат: or-glm-5.3 на «среднем двух чисел» потратила **7778 токенов рассуждения**, при лимите 3000 выдала пустой ответ (finish_reason: length); вызов стоил \$0,045. Файл сохранён: это **артефакт измерения**, а не ответ модели — тема сегмента «откуда мы знаем».

Та же арифметика, спрятанная внутри задачи

Та же арифметика — но *внутри* правдоподобной работы: в промптах нет ни слова про пределы, размеры и корректность. По одной генерации на пару «модель × задача».

спрятанная ловушка	qwen3.8	GPT-5.1	Claude Sonnet 5
середина отрезка	чисто	чисто	ошибка
сумма $1 \dots n$	чисто	ошибка	ошибка
факториал	чисто	ошибка	ошибка
те же три впрямую	не все чисто	чисто	чисто

Ответ Claude на «середину отрезка» — $(x1 + x2) / 2.0$: возвращает `double`, *выглядит* аккуратно, а складывается в `int` и переполняется **до** деления. Запуском: `midpoint(2e9, 2e9)` даёт `-147483648.0` при истинных `2000000000.0`.

Как читать код стандартной библиотеки

- Имена с подчёркиваниями (`__lower_bound`, `__half`) — внутренние имена реализации: стандарт резервирует их за библиотекой, чтобы не столкнуться с именами и макросами пользователя. Снаружи — `std::lower_bound`.
- `_ForwardIterator` — параметр шаблона; читайте его как «указатель». `__comp` — компаратор, по умолчанию «меньше».
- Из всего этого для нас важно одно: **на чём цикл считает середину** — на индексах или на длине.
- Дальше — семь реализаций одной функции. Читаем тело цикла, сигнатуру — по диагонали.

пояснение один раз, как в дне 1; дальше по коду эти вещи не разъясняются.

libstdc++: std::__lower_bound

```
1467     while (__len > 0)
1468     {
1469         _DistanceType __half = __len >> 1;
1470         _ForwardIterator __middle = __first;
1471         std::advance(__middle, __half);
1472         if (__comp(__middle, __val))
1473         {
1474             __first = __middle;
1475             ++__first;
1476             __len = __len - __half - 1;
1477         }
1478         else
1479             __len = __half;
1480     }
```

Делится **длина**, а не сумма индексов: складывать нечего. Выхода по равенству нет — это точка раздела, а не «найти элемент».

/usr/include/c++/13/bits/stl_algobase.h (g++ 13.3.0): на слайде строки 1467-1482, функция std::__lower_bound целиком — 1456-1482 (C1); вызов из std::lower_bound — 1495-1509.

libc++: тот же цикл, та же идея

```
32 __lower_bound(_Iter __first, _Sent __last, const _Type& __value, _Comp& __comp, _Proj& __proj) {
33     auto __len = _IterOps<AlgPolicy>::distance(__first, __last);
34
35     while (__len != 0) {
36         auto __l2 = std::__half_positive(__len);
37         _Iter __m = __first;
38         _IterOps<AlgPolicy>::advance(__m, __l2);
39         if (std::__invoke(__comp, std::__invoke(__proj, *__m), __value)) {
40             __first = ++__m;
41             __len -= __l2 + 1;
42         } else {
43             __len = __l2;
44         }
45     }
46     return __first;
```

`__half_positive(__len)` — та же длина: вторая библиотека пришла к приёму независимо (тот же приём — у Слотина).

`/usr/lib/llvm-18/include/c++/v1/__algorithm/lower_bound.h`, строки 32-47 (сигнатура — 30-31) — C2; он же — SL, стр. 7.

glibc bsearch: цикл без арифметики индексов

```
26  while (__nmemb)
27  {
28      __p = (const void *) (((const char *) __base) + ((__nmemb >> 1) * __size));
29      __comparison = (*__compar) (__key, __p);
30      if (__comparison == 0)
```

// строки 31–40 опущены: ветка «найдено» с #pragma — к середине отношения не имеет

```
41      if (__comparison > 0)
42      {
43          __base = ((const char *) __p) + __size;
44          --__nmemb;
45      }
```

Третья библиотека — независимо к тому же.

Проверено 16.09.2026: утверждение, что в glibc стоит $(__l + __u) / 2$, для master неверно; в установленном здесь glibc 2.39 цикл ещё считает по индексам.

bits/stdlib-bsearch.h (master, зеркало bminor/glibc): на слайде строки 26–30 и 41–46, цикл целиком — 20–50 — СЗ.

Java: Arrays.binarySearch0 — починка Блоха внутри JDK

```
1704     while (low <= high) {
1705         int mid = (low + high) >>> 1;
1706         int midVal = a[mid];
1707
1708         if (midVal < key)
1709             low = mid + 1;
1710         else if (midVal > key)
1711             high = mid - 1;
1712         else
1713             return mid; // key found
1714     }
```

// строки 1703-1704 — инициализация low и high

// строки 1703-1704 — инициализация low и high

>>> — сдвиг без знака: сумма может переполниться, но после >>> 1 бывший знаковый бит — не знак. Починка Блоха внутри JDK.

OpenJDK, java.base/java/util/Arrays.java, binarySearch0(int[],int,int,int): на слайде строки 1704-1714, метод

целиком — 1699-1716 (master, прочитано 16.09.2026).

CPython bisect: переполнение, названное по имени

```
74     while (lo < hi) {
75         /* The (size_t)cast ensures that the addition and subsequent division
76            are performed as unsigned operations, avoiding difficulties from
77            signed overflow. (See issue 13496.) */
78         mid = ((size_t)lo + hi) / 2;
79         assert(mid >= 0);
```

Приведение к `size_t` поставлено *специально*, и комментарий прямо говорит зачем — со ссылкой на [issue 13496](#). То же выражение, что в JDK, только скобка стоит в другом месте.

CPython, `Modules/_bisectmodule.c`, `internal_bisect_right`, строки 74-79 (main, прочитано 16.09.2026): комментарий — 75-77, присваивание — 78. Тот же комментарий над `internal_bisect_left` — строки 255-258 (C4: 77-78 и 257-258).

Go sort.Search: сложение индексов — но беззнаковое

```
58 func Search(n int, f func(int) bool) int {  
59     // Define f(-1) == false and f(n) == true.  
60     // Invariant: f(i-1) == false, f(j) == true.  
61     i, j := 0, n  
62     for i < j {  
63         h := int(uint(i+j) >> 1) // avoid overflow when computing h
```

// строки 64–64 опущены: тот же комментарий стоит и над строкой с h

```
65         if !f(h) {  
66             i = h + 1 // preserves f(i-1) == false  
67         } else {  
68             j = h // preserves f(j) == true  
69         }
```

// строки 64–64 — тот же комментарий стоит и над строкой с h

Индексы всё-таки складывают — но в uint: переполнение становится определённым.

Мораль: ни одна библиотека не пишет $(low + high) / 2$

реализация	как считает середину	строка
libstdc++	<code>__len >> 1</code> — длина	<code>stl_algobase.h:1459</code>
libc++	<code>__half_positive(__len)</code> — длина	<code>lower_bound.h:36</code>
glibc	<code>__nmemb >> 1</code> — длина	<code>stdlib-bsearch.h:28</code>
Java	<code>(low + high) >>> 1</code> — без знака	<code>Arrays.java:1705</code>
CPython	<code>((size_t)lo + hi) / 2</code>	<code>_bisectmodule.c:78</code>
Go	<code>int(uint(i+j) >> 1)</code>	<code>search.go:63</code>

Строчка из учебника в продакшене не встречается: три библиотеки делят *длину*, три складывают индексы в беззнаковом типе.

номера строк — из перечисленных выше версий (g++ 13.3.0, llvm-18, master/upstream, прочитано 16.09.2026).

Rust объясняет сам себя — и разменивает лучший случай

```
2996 // This loop intentionally doesn't have an early exit if the comparison
2997 // returns Equal. We want the number of loop iterations to depend *only*
2998 // on the size of the input slice so that the CPU can reliably predict
2999 // the loop count.
3000 while size > 1 {
3001     let half = size / 2;
3002     let mid = base + half;
3003
3004     let cmp = f(unsafe { self.get_unchecked(mid) });
3005
3006     // Binary search interacts poorly with branch prediction, so force
3007     // the compiler to use conditional moves if supported by the target
3008     // architecture.
3009     base = hint::select_unpredictable(cmp == Greater, base, mid);
3010
3011     // ...
3012 }
```

Выхода по равенству **нет сознательно** (2996–2999), а `select_unpredictable` (3012) — это и есть безветвенный выбор, `mov` вместо перехода.

Замер №1: мы против `std::lower_bound`

демо

n	2^{10}	2^{16}	2^{20}	2^{24}
наш цикл, нс на запрос	28,0	52,9	94,0	430,4
<code>std::lower_bound</code> , нс	26,6	51,0	91,7	430,5
сравнений на запрос, <i>оба</i>	10,000	16,000	20,000	24,000

Меряем **стоимость одного поиска**: поток 10^7 запросов, каждый следующий зависит от ответа на предыдущий — иначе цикл выбросили бы компилятор и процессор. Каждый ответ сверяется с `std::lower_bound`: не проверился — замер не сделан.

Ожидание вслух до запуска: «библиотека должна выиграть, она же оптимизирована». Она выиграла — на 2-5 %, а на 2^{24} они *сравнились*. Сравнений *ровно* поровну, разница 0,000. Быстрее нельзя?

стенд M1 (Zen 5, taskset -c 1, g++ 13.3.0, -O2 -std=c++17 -falign-loops=64); demo/bench-binsearch/results.md,

«Эксперимент 1»; строки B1, B2 в numbers.md.

Куда уходит время: один слайд perf

perf record & perf report, $n = 10^6$ --- SL, стр. 8

35:	mov	%rax,%rdx	
0,52		sar	%rdx
0,33		lea	(%rsi,%rdx,4),%rcx
4,30		cmp	(%rcx),%edi
65,39		↓ jle	b0
9,32		lea	0x4(%rcx),%rsi
1,37		test	%rax,%rax
			↑ jg 35

a[(l + r) / 2] < x?
--- условный переход

Больше двух третей времени — не на сравнении и не на загрузке, а на **условном переходе**. Каждый шаг поиска по случайному запросу — «монетка»: предсказатель угадывает $\approx 50\%$, промах стоит сброса конвейера в **15-20 тактов** (SL 11, 14). Наши счётчики на M1: **0,40-0,43 промаха предсказания на шаг** поиска — $3,98 \cdot 6,79 \cdot 8,58 \cdot 10,04$ на запрос при $n = 2^{10}, 2^{16}, 2^{20}, 2^{24}$.

SL, стр. 8 — по данным автора, на его железе (Zen 2 @ 2 ГГц, DDR4-2667, SL 5); SL 11, 14 — длина конвейера; наши счётчики

Безветвенный: та же задача без перехода

Три шага Слотина (SL 16 $\rightarrow$ 17 $\rightarrow$ 19), все три здесь:

1. «база + длина» вместо пары границ $[lo, hi]$;
2. `len -= half` — длина уменьшается до сравнения, итераций ровно $\lceil \log_2 n \rceil$, и они не зависят от данных;
3. `if` заменён арифметикой — компилятор ставит `cmov`.

«Безветвенный» — не «без сравнений»: сравнение осталось, исчез *переход*.

```
1 int *base = a;           // 1. база и длина вместо [lo, hi)
2 size_t len = n;
3 while (len > 1) {
4     size_t half = len / 2;
5     len -= half;          // 2. длина падает до сравнения
6     base += (base[half - 1] < x) // 3. сравнение -> арифметика,
7         * half;           // компилятор ставит cmov
8 }
9 return base + (*base < x);
```

Замер №2: безветвенный против библиотеки демо

n	2^{10}	2^{13}	2^{16}	2^{20}	2^{25}
безветвенный (cmov), нс	20,5	25,9	43,7	102,2	1 030,3
std::lower_bound, нс	26,6	35,7	51,0	91,7	628,3
std / безветвенный	1,29	1,38	1,17	0,90	0,61

Заявка Слотина в выводах доклада: «Branchless binary search: **3x** on small arrays» — по данным автора, на его железе (Zen 2 @ 2 ГГц, DDR4-2667). У нас на малых n — 1,3–1,4 $\times$, а после 2^{18} безветвенный *проигрывает*.

Почему не 3 $\times$: меряем не ту величину. Тот же бинарник, те же ключи, но запрос больше не зависит от ответа на предыдущий — и выигрыш безветвенного при 2^{16} растёт **с 1,17 $\times$ до 2,27 $\times$** , а при 2^{10} — с 1,29 $\times$ до 2,36 $\times$. Мы меряем **задержку одного зависимого поиска**, в докладе же — **пропускная способность** потока независимых запросов. Это разные величины, а не разные результаты.

«Why worse on larger arrays?» — SL, стр. 20–21.

Причина — SL 22: **предвыборка через спекуляцию.**

- Ветвистая версия предсказывает переход — и, пусть неверно, начинает тянуть из памяти обе половины ещё до того, как сравнение досчитано.
- Безветвенная честно ждёт `stov` и ходит в память строго последовательно: она-то знает, что пойдёт именно туда.
- Убрав ветвление, мы убрали и бесплатную предвыборку.

Неправильно предсказанная ветвь всё-таки сходила в память, и данные уже в кэше; правильный безветвенный код в память не сходил.

Замер №3: предвыборка, возвращённая вручную

демо

```
150     size_t half = len >> 1;
151     __builtin_prefetch(base + half - 1);
152     __builtin_prefetch(base + 2 * half - 1);
153     len -= half;
```

Безветвенный цикл лишился бесплатной предвыборки от спекуляции. Просим её руками (SL 23); half посчитан строкой выше, адреса видны целиком.

n	2^{10}	2^{16}	2^{20}	2^{24}
ветвистый, нс	28,6	54,3	97,0	589
безветвенный, нс	21,1	47,9	111,3	1116
безветвенный + prefetch, нс	24,3	50,9	117,2	1125

Предвыборка **медленнее** обычного безветвенного на каждом размере — от +3 до +15 %, и на всех тринадцати размерах до 2^{22} тоже. У Слотина этот приём проигрыш *чинит*.

Почему у нас — нет?

```
129     size_t half = len >> 1;
130     len -= half;
131     __builtin_prefetch(&base[len / 2 - 1]);           // middle of the left half
132     __builtin_prefetch(&base[half + len / 2 - 1]);    // middle of the right half
```

Одна перестановка: `len -= half` **до** предвыборки — тогда `len` уже следующая длина, и адреса указывают на две возможные пробы *следующей* итерации.

	было: просим		стало: просим		прочтут на самом деле	
len	[a]	[b]	[a]	[b]	влево	вправо
1024	511	1023	255	767	255	767
512	767	1023	639	895	639	895
256	895	1023	831	959	831	959

[a] было `base + half - 1` — проба *этой* же итерации, её и так читают строкой ниже; [b] — последний элемент диапазона, он не проба никогда. Правые столбцы совпадают с настоящей следующей пробой в точности.

С правильными адресами: **39,0** нс на 2^{16} (−10,8 %), **75,5** на 2^{20} (−26,1 %), **461,8** на 2^{24} (−39,4 %). **Проигрыш безветвленного исчез.**

Замер №3 заново: та же таблица, правильные адреса

демо

```
129     size_t half = len >> 1;
130     len -= half;
131     __builtin_prefetch(&base[len / 2 - 1]);           // middle of the left half
132     __builtin_prefetch(&base[half + len / 2 - 1]);    // middle of the right half
```

Тот же приём, тот же цикл, те же ключи и тот же прогон, что двумя слайдами раньше. Изменились *только* два адреса.

n	2^{10}	2^{16}	2^{20}	2^{24}
ветвистый, нс	28,6	54,3	97,0	589
безветвенный, нс	21,1	47,9	111,3	1116
безветвенный + prefetch, нс	21,2	39,9	77,7	697
было (слайд с вопросом)	24,3	50,9	117,2	1125

Первые две строки — *те же самые числа*, что и на слайде с вопросом: сравнивать можно построчно. Предвыборка из проигрыша (+15 % на 2^{16}) стала выигрышем: −17 % на 2^{16} , −30 % на 2^{20} , −38 % на 2^{24} . **Проигрыш безветвенного она чинит — ровно как у Слотина.**

Спекуляция и предвыборка глубже

- SL 48–49: предвыборка меняет **пропускную способность на задержку**. Там, где полосы памяти и так не хватает — например, в многопоточном коде — это работает плохо.
- SL 70: спайки в конце графика — переполнение L1 TLB.
- Наши ряды сняты на **одном** ядре под taskset, и ряд Эйтцингера, который будет во второй паре, — тоже. Цена многопоточности — отдельный замер, которого в листе чисел нет.

Тестировался только однопоточный поиск.

SL, стр. 48, 49, 70 — по данным автора, на его железе (Zen 2 @ 2 ГГц, DDR4-2667, SL 5).

Цена решения Rust в API

- `binary_search` **не гарантирует** самое левое совпадение среди равных: цикл без раннего выхода возвращает *какое-то* совпадение.
- Для детерминированного ответа в Rust есть `partition_point` — тот же `lower_bound`, но названный своим настоящим именем.

Там, где выбор производительности меняет *ответ*, он обязан быть в документации, а не в комментариях к циклу. Решение, принятое ради предсказуемости числа итераций, стало видно пользователю в спецификации: API важнее скорости.

`library/core/src/slice/mod.rs: binary_search_by` (комментарии — 2996-2999) и `partition_point`; SL, стр. 16-19.

Четыре модели — одна формула

$$(a \& b) + ((a \wedge b) \gg 1)$$

- **4 из 6** безопасных ответов на «среднее двух чисел» дали одну и ту же битовую формулу (L7).
- Приём из *Hacker's Delight*: сумма без переносов — это $\wedge$, переносы — $\&$, сдвинутые на бит. Переполниться нечему.
- В курсе эта формула уже звучала; здесь она появилась сама.
- **А чем не $((\text{unsigned})lo + (\text{unsigned})hi) \gg 1$?** — **Ничем: оба корректны.** Беззнаковая арифметика определена по модулю, сумма в `unsigned` помещается — ровно так делает Java своим `>>>`. С разностью удобнее: тип не менять, и работает для знаковых и для итераторов.

Интерполяционный поиск

- Идея: вместо середины — оценка положения ключа,
$$i \approx lo + (x - a[lo]) \frac{hi - lo}{a[hi] - a[lo]}.$$
- Кнут, 6.2.1: на равномерных ключах ожидаемо $O(\log \log n)$ проб.
- В худшем случае — $O(n)$: оценка может указать на самый край, и отрезок уменьшится на единицу.
- Туда же, куда и RocksDB с слайда выше: приём имеет смысл там, где известен закон распределения ключей.

Где вообще ищут

- `git bisect` — бинарный поиск по истории коммитов: n — число коммитов между «работало» и «не работает», проверяющая функция — сборка и тест.
- Блоки SSTable в LevelDB/RocksDB: поиск по индексному блоку — тот же `lower_bound`. В 2026 году RocksDB добавила туда интерполяционный поиск: 335 749 → 366 598 оп/с (+9,2%) на `readrandom`, один уровень, равномерные ключи — по данным авторов, на их тесте; для перекошенных ключей выигрыш не заявляется.
- `std::lower_bound` внутри токенизатора и внутри `top-p` из дня 1: там сортировка по накопленной сумме, здесь — поиск по ней.

Троичный поиск, и почему он хуже

- Два сравнения на шаг, основание логарифма 3: шагов $\log_3 n$, но каждый дороже.
- Смена основания: $\log_3 n \cdot 2 = \frac{2}{\log_2 3} \log_2 n \approx 1,26 \log_2 n$ — на 26 % больше работы, чем у двоичного.
- Меньше итераций — не значит быстрее. Ровно та ошибка, с которой мы начали день 1.

$\log_2 3 \approx 1,585$, значит $2/\log_2 3 \approx 1,26$.

арифметика смены основания логарифма; сюжетная рифма с Александреску (лекция 1).

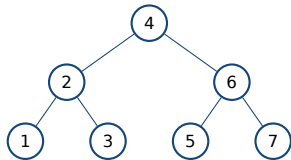
Массив неподвижен

Всё, что мы делали до сих пор, требует **статического** отсортированного массива. Одна вставка в середину — $O(n)$.

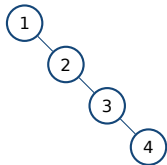
БДП — двоичное дерево поиска: то же деление пополам, но переживающее вставку. **Дерево указателей** — узлы в куче, дети — указатели (не массив, — 2.5).

- **Свойство БДП:** для любого узла x — всё левое поддереву $\leq \text{key}[x]$, всё правое $\geq \text{key}[x]$.
- Равные ключи: договариваемся один раз — у нас они уходят *вправо*. Пока не договорились, «поиск» не определён.
- Все операции — $O(h)$, где h — высота. **Не** $O(\log n)$: это разные вещи, и следующий слайд показывает, насколько.

Одни и те же ключи — две разные высоты



вставляли 4, 2, 6, 1, 3, 5, 7 — высота 2



вставляли 1, 2, 3, 4, ... — высота $n - 1$

Худшее **дерево** БДП — **вырожденное**, «палка» высоты $n - 1$ (справа): наказуема **форма**, а не вход — отсортированный массив лишь самый частый способ её получить. Та же беда, что у `std::sort` в дне 1, и по той же причине.

Кнут о случайном порядке вставок: «tree search will require only about $2 \ln N \approx 1,386 \lg N$ comparisons... degenerate trees are very rare» — на 39 % больше сравнений, и бесплатно.

цитата — Кнут, т. 3, § 6.2.2, печатная с. 430 (C17 в `sources/knuth/citations.md`); рисунок — MS «Примеры деревьев

поиска».

Операции БДП: все — спуск по одному пути

Операция	Как	Время
Tree-Search	идти влево/вправо по сравнению	$O(h)$
Tree-Min / Max	до упора влево / вправо	$O(h)$
in-order обход	левое, узел, правое	$O(n)$
Tree-Successor	минимум правого поддерева, а если его нет — вверх, пока идём справа	$O(h)$
Tree-Insert	спуск с «отстающим» указателем	$O(h)$
Tree-Delete	три случая — следующий слайд	$O(h)$

In-order обход даёт отсортированный порядок: дерево умеет то, чего не умеет хеш-таблица — главный ответ на «зачем тогда деревья» в конце пары.

MS «Поиск», «Поиск — итеративный», «Вывод ключей в упорядоченном порядке», «Минимум/максимум», «Следующий/предшествующий элемент», «Вставка»; CLRS § 12.1–12.3.

Удаление: три случая, и третий — самый тонкий

- **Нет детей** — отцепить.
- **Один ребёнок** — поставить ребёнка на своё место.
- **Два ребёнка** — взять *преемника* (минимум правого поддеревя; у него заведомо нет левого ребёнка), поставить его на место удаляемого.

И здесь развилка: преемника можно **скопировать** (перенести ключ) или **перевязать** (переставить сам узел с указателями). Копирование короче — и ломает все внешние указатели на узел, ключ которого подменили. Поэтому CLRS 3-го изд. переписал процедуру через Transplant, и в конспекте — одна версия, тоже через Transplant.

Выбор из учебника, видимый в ABI библиотеки

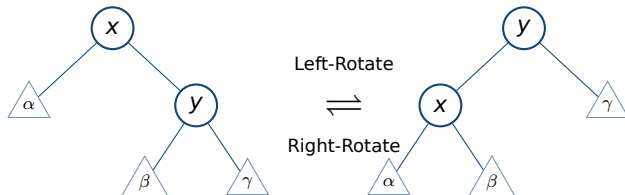
```
95 // (2) when a node being deleted has two children its successor node
96 // is relinked into its place, rather than copied, so that the only
97 // iterators invalidated are those referring to the deleted node.
```

Это комментарий в `std::tree.h` — заголовке, из которого собран ваш `std::map`. Учебник даёт **два** способа удалить узел с двумя детьми: *скопировать* ключ преемника в удаляемый узел — или вынуть сам узел-преемник и *перевязать* его на это место. `libstdc++` выбрал второй.

Почему — не «так красивее». Стандарт обещает, что `erase` портит итераторы и ссылки **только на удаляемый элемент**: «References and iterators to the erased elements are invalidated. Other references and iterators are not affected». Скопируй библиотека ключ — сломались бы ещё и итераторы на *узел-преемник*, который пользователь не удалял. Это было бы нарушением обещания, поэтому способ из учебника ей просто запрещён.

Первое за день место, где выбор реализации продиктован гарантией стандарта.

Поворот: три указателя, $O(1)$, порядок не меняется



in-order слева: $\alpha x \beta y \gamma$ — in-order справа: $\alpha x \beta y \gamma$

Порядок не изменился ни на один ключ — значит, свойство БДП цело, и проверять его после поворота не нужно. Изменились только высоты, и только локально. Время — $O(1)$: константное число присваиваний, независимо от n .

Тот же поворот в libstdc++ — три присваивания и правка родителя

```
136  _Rb_tree_node_base* const __y = __x->_M_right;
137  __x->_M_right = __y->_M_left;
138  if (__y->_M_left != 0)
139      __y->_M_left->_M_parent = __x;
140  __y->_M_parent = __x->_M_parent;
141  if (__x == __root)
142      __root = __y;
143  else if (__x == __x->_M_parent->_M_left)
144      __x->_M_parent->_M_left = __y;
145  else
146      __x->_M_parent->_M_right = __y;
147  __y->_M_left = __x;
148  __x->_M_parent = __y;
```

Первые две строки и последние две — ровно α , β , γ с рисунка; между ними — бухгалтерия родителя, которой в учебнике нет.

libstdc++-v3/src/c++98/tree.cc (gcc, ветка master, прочитано 17.09.2026), local_Rb_tree_rotate_left — строки 132-151; на слайде 136-150. Код C5 в numbers.md.

AVL: два бита на узел — и высота перестаёт быть проблемой

Определение. Дерево сбалансировано, если у *любого* узла высоты левого и правого поддеревьев отличаются не больше чем на 1. Баланс узла $\in \{-1, 0, +1\}$.

Кнут, § 6.2.3 — почему не «держать дерево оптимальным»:

«Perhaps we could devise an algorithm that keeps the tree optimum at all times; but unfortunately that seems to be very difficult.»

*«A very pretty solution... was discovered in 1962 by two Russian mathematicians, G. M. Adelson-Velsky and E. M. Landis... Their method requires only **two extra bits per node**, and it never uses more than $O(\log N)$ operations to search the tree or to insert an item.»*

Два бита — это вся цена балансировки.

Почему высота AVL — это $O(\log n)$: худшее дерево — Фибоначчи

m_h — **наименьшее** число узлов AVL-дерева высоты h (в рёбрах).
Худшее дерево: корень, под ним поддеревья высот $h-1$ и $h-2$.

$$m_h = m_{h-1} + m_{h-2} + 1, \quad m_0 = 1, \quad m_1 = 2 \quad \Rightarrow \quad m_h = F_{h+3} - 1$$

h	0	1	2	3	4	5
m_h	1	2	4	7	12	20

Индукцией по $\varphi^2 = \varphi + 1$: $F_h \geq \varphi^{h-2}$, и $n \geq m_h$ даёт $h = O(\log n)$.

Теорема А (Адельсон-Вельский и Ландис, у Кнута): высота лежит между $\lg(n+1)$ и $1,4404 \lg(n+2) - 0,3277$ — «*never more than 45 percent longer than the optimum*».

Кнут, т. 3, § 6.2.3, с. 460, теорема А (C19a); у него высота в уровнях — на единицу больше, чем в лемме выше.

Вставка в AVL: четыре случая, из них два — зеркала

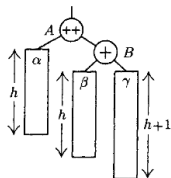
Случай	Куда вставили	Чиним
LL	в левое поддереву левого сына	один поворот вправо
RR	в правое поддереву правого сына	один поворот влево
LR	в правое поддереву левого сына	влево в сыне, вправо в узле
RL	в левое поддереву правого сына	вправо в сыне, влево в узле

При вставке хватает одной перебалансировки: поворот возвращает поддереву прежнюю высоту, и выше по пути ничего не меняется. При удалении не так — поворотов может понадобиться до $O(\log n)$. Двойной поворот *выводим*, а не запоминаем картинкой: LR — это Left-Rotate в сыне плюс Right-Rotate в узле, оба — те же, что в 2.2.

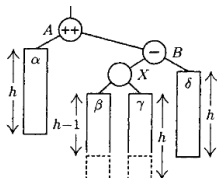
Где этого кода нет. Ни в `std::map`, ни в `java.util.TreeMap`, ни в ядре Linux AVL-деревя нет — везде красно-чёрное. Поэтому AVL мы строим на доске, а читаем — КЧ.

MS «Вставка в AVL-дереву», «Балансировка при вставке»; Кнут § 6.2.3, алгоритм A; CLRS задача 13-3(a); про библиотеки —

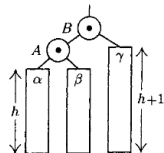
Перебалансировка при вставке — все четыре дерева сразу



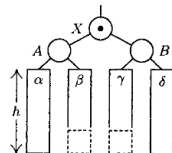
Случай 2



до: левый ряд — случай 1 (LL: баланс A — $++$, сына B — $+$), правый — случай 2 (LR: B — $-$, новый узел пришёл в γ , пунктир)



Случай 2



после: один поворот вправо в A и двойной (в B , затем в A); RR , RL — зеркальные

Зачем подписаны высоты: после поворота максимальная высота поддеревя равна той, что была до вставки ($h + 1$) — значит выше по пути балансы не изменились. «Хватает одной перебалансировки» — не наблюдение, а подсчёт по этой картинке.

Удаление из AVL: стек пар (P_k, D_k) — и почему поворотов много

- ▶ Если $k = 0$, завершить выполнение алгоритма.
- ▶ $bal(P_k) = D_k \implies bal(P_k) \leftarrow 0, k \leftarrow k - 1$, повторить корректировку для нового k .
- ▶ $bal(P_k) = 0 \implies bal(P_k) \leftarrow -D_k$, завершить выполнение алгоритма.
- ▶ $bal(P_k) = -D_k \implies$ требуется перебалансировка:
 - ▶ Аналогично добавлению: рассматриваем поддерево, в котором не удаляли элемент, как дерево с добавленным элементом.
 - ▶ Добавляется третий случай, аналогичный первому но с высотой β равной $h + 1$.
- ▶ Важное отличие от добавления элемента: может потребоваться $\log n$ поворотов, тогда как для добавления не требуется больше 1.

P_k — узлы пути вниз до удаляемого, D_k — направление, куда мы там шли. Удалили — и корректируем баланс, поднимаясь по стеку:

- $bal(P_k) = D_k$ — наклонён туда, откуда ушёл узел: $bal := 0$, **высота упала** $\Rightarrow$ идём выше;
- $bal(P_k) = 0$ — стал наклонён против D_k , высота та же: **останов**;
- $bal(P_k) = -D_k$ — **перебалансировка**; здесь у удаления появляется *третий* случай: среднее поддерево β может иметь высоту $h + 1$.

Отсюда цена. При вставке поворот возвращает поддереву прежнюю высоту — цикл кончается. При удалении высота падает, и идти надо до корня: поворотов $O(\log n)$ вместо ≤ 2 .

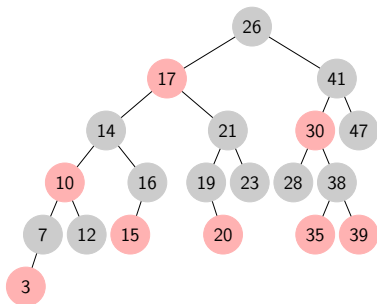
Красно-чёрные: пять свойств — из комментария в ядре Linux

```
15  /*
16   * red-black trees properties: https://en.wikipedia.org/wiki/Rbtree
17   *
18   * 1) A node is either red or black
19   * 2) The root is black
20   * 3) All leaves (NULL) are black
21   * 4) Both children of every red node are black
22   * 5) Every simple path from root to leaves contains the same number
23   *    of black nodes.
```

Свойство 5 — **чёрная высота** $bh(x)$: чёрных узлов на пути вниз, не считая сам x . Один общий фиктивный NIL на всё дерево — так удобнее и в учебнике, и в коде.

`lib/rbtree.c` (Linux, ветка master, прочитано 17.09.2026), шапка файла — строки 15–32; на слайде 15–23. Цитата E7 в `numbers.md`. Пример дерева — CLRS рис. 13.1.

Пример: то же дерево, что на обоих наборах слайдов



Считаем:

- **(4)** у каждого красного — оба сына чёрные: 17, 10, 3, 15, 20, 30, 35, 39 — проверить каждого;
- **(5)** из корня 26 до *любого* NIL — ровно три чёрных узла, если сам 26 не считать. Это и есть чёрная высота $b_h = 3$.

Самый длинный путь —

$26 \rightarrow 17 \rightarrow 14 \rightarrow 10 \rightarrow 7 \rightarrow 3 \rightarrow \text{NIL}$, шесть рёбер; самый короткий — $26 \rightarrow 41 \rightarrow 47 \rightarrow \text{NIL}$, три. Отношение $6/3 = 2$: свойства 4 и 5 дают «не более двух подряд», и пара «красный $\rightarrow$ чёрный» эту границу *достигает* — без них оценки бы просто не было.

Проверка сверху: $n = 20$, потолок $2\lg(20 + 1) = 8,6$ при реальной высоте 6. Оценка — не обещание, а потолок.

Высота — и её доказательство там же, следующим абзацем

25 * 4 and 5 give the $O(\log n)$ guarantee, since 4 implies you cannot have two
26 * consecutive red nodes in a path and every red node is therefore followed by
27 * a black. So if B is the number of black nodes on every simple path (as per
28 * 5), then the longest possible path due to 4 is $2B$.

Это всё доказательство. Формально — две строки: поддереву узла x содержит не меньше $2^{bh(x)} - 1$ внутренних узлов (индукция по высоте), а по свойству 4 половина пути чёрная, то есть $bh(\text{корень}) \geq h/2$. Отсюда $n \geq 2^{h/2} - 1$ и

$$h \leq 2\lg(n + 1).$$

lib/rbtree.c, строки 25-28 — цитата E7; формальный вариант — CLRS § 13.1, лемма 13.1.

Вставка: три случая — и они же в libstdc++

Красим новый узел в **красный** (свойство 5 не трогаем) и, пока отец красный, смотрим на *дядю*:

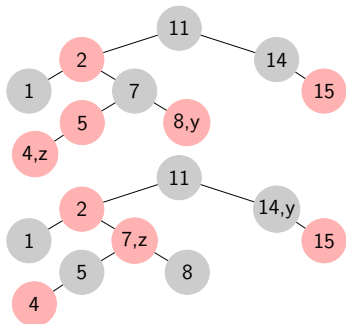
1. дядя **красный** — перекрасить отца и дядю в чёрный, деда в красный, подняться на два уровня;
2. дядя чёрный, *z* — правый сын — поворот в отце, переходим к 3;
3. дядя чёрный, *z* — левый сын — перекрасить и повернуть в деде.

Корень в конце красим в чёрный. **Поворотов не больше двух** — вот почему вставка в КЧ дешевле, чем кажется.

```
249     if (__x == __x->M_parent->M_right)
250     { __x = __x->M_parent;
251       local_Rb_tree_rotate_left(__x, __root); }
252     __x->M_parent->M_color = _S_black;
253     __xpp->M_color = _S_red;
254     local_Rb_tree_rotate_right(__xpp, __root);
```

libstdc++-v3/src/c++98/tree.cc, `_Rb_tree_insert_and_rebalance` — с 194; повороты в строках 252, 256, 274, 278 (C5).

Вставка, случай 1: дядя красный — поворотов нет



Вставляем 4 в дерево CLRS: 11, 2, 14, 1, 7, 15, 5, 8. Новый узел красный — нарушено **свойство 4**: отец 5 красный. Дядя у — это 8, он тоже **красный**. Тогда

$color[p[z]] := BLACK, \quad color[y] := BLACK,$
 $color[p[p[z]]] := RED, \quad z := p[p[z]].$

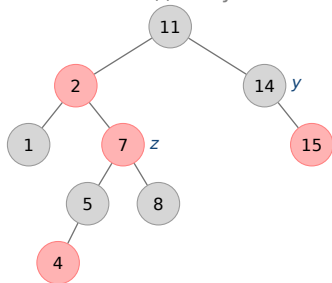
Почему это законно. Свойство 5 сравнивает пути от *одного* узла: ниже 7 все пути стали длиннее по чёрному на единицу, внутри поддеревьев 5 и 8 — ничего. Долг переехал с 4 на 7: z уехал на два уровня вверх, и while поедет выше.

OS «Вставка. Иллюстрация», с. 23 (картинка — из архива 2010 г.); процедура — CLRS §13.3, RB-Insert-Fixup, случай 1 (y — дядя); тот же пример — MS «Вставка», plan.md, п. 2.4.

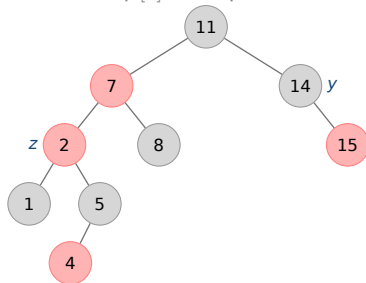
Вставка, случай 2: z — правый сын

После случая 1 долг уехал наверх: $z = 7$ красный, отец 2 красный, дядя $y = 14$ чёрный, и z — **правый** сын.

было: вход в случай 2



стало: $z := p[z]$, поворот влево в 2



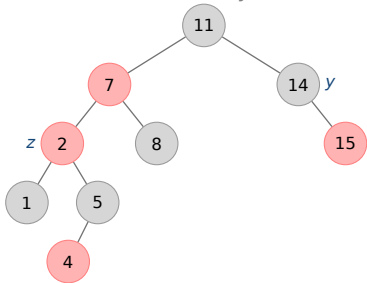
Метка z переехала с 7 на 2, а 7 поднялся и стал ей отцом. **Свойство 4 он не починил:** $z = 2$ красный и отец 7 красный — два красных подряд на месте. Случай 2 только *меняет форму*: делает z левым сыном, то есть переводит задачу в случай 3.

CLRS §13.3, случай 2 (рис. 13.46→в); тот же пример — OS «Вставка. Иллюстрация», с.23-24 (архив 2010 г.); код — tree.cc, повороты 252, 256.

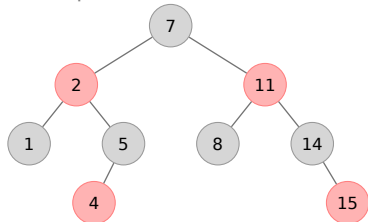
Вставка, случай 3: z — левый сын, и это финал

Теперь $z = 2$ — **левый** сын, отец 7 красный, дед 11 чёрный. Красим отца в чёрный, деда в красный, поворот вправо в деде.

было: вход в случай 3



стало: дерево законно, цикл кончился



Над поддеревом снова **чёрный** узел — бывший отец 7, и оба его сына красные: свойство 4 цело. Чёрная высота поддерева не изменилась, значит выше ничего не поехало, и **while не продолжается**.

Итог вставки: перекрасок $O(\log n)$, а **поворотов за всю вставку не больше двух** — один в случае 2 и один в случае 3.

Удаление — в двух предложениях. И чем его проверять

- Если удалили **чёрный** узел, на его заместителе оказывается «лишняя чернота»: свойство 5 нарушено ровно на этом пути.
- Её *сгоняют вверх* четырьмя случаями, разбираемыми по цвету брата и цветам его детей, пока она не дойдёт до корня или не погасится красным узлом.

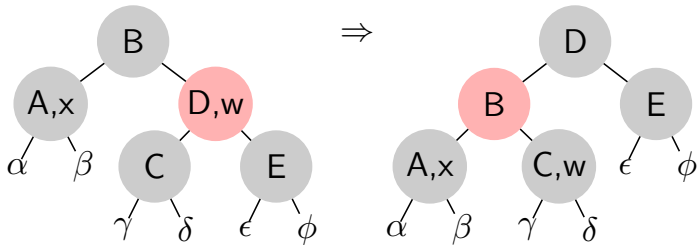
Все четыре случая — на следующих двух слайдах.

Полезная мелочь для лабораторной. В `libstdc++` есть готовый проверяльщик инвариантов — `_Rb_tree::__rb_verify()` (`stl_tree.h`, объявление 1401, тело 2569). Он считает чёрную высоту и проверяет, что у красного узла нет красных детей: ровно то, что вы напишете сами в тесте к своей реализации.

Удаление, случай 1: брат w — красный

Удаляемого узла на схеме нет: он уже *убран*, а x — тот, кто занял его место. Раз удалили *чёрный* узел, на x повис лишний *чёрный*, и свойство 5 нарушено ровно на этом пути. Процедура ничего не удаляет — она гасит этот долг.

Случай 1. Брат w (на схеме D) — **красный**. Красим w в *чёрный*, отца в *красный*, поворот в отце. Долг остаётся, но брат у x стал *чёрным* — а это уже случай 2, 3 или 4.

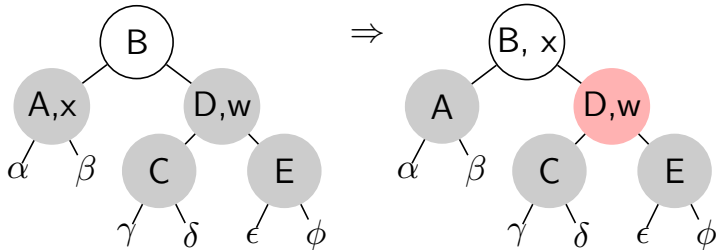


OS «Удаление. Иллюстрация», с. 28 (картинки — из архива 2010 г.); нумерация и переходы — CLRS § 13.4, RB-Delete-Fixup;

все случаи нарисованы для « x — левый сын».

Удаление, случай 2: брат чёрный и оба его сына чёрные

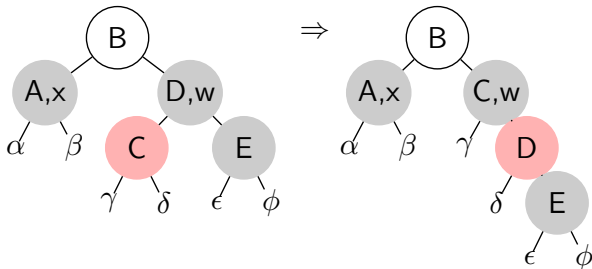
Случай 2. w чёрный, оба его ребёнка чёрные: $color[w] := RED$, $x := p[x]$. Ни одного поворота — лишнюю черноту снимают с x и вешают на отца, долг уехал на уровень выше. Это единственный случай, который идёт вверх.



Белые кружки (B, x) — цвет «любой, какой был»: процедура его не читает, и это часть доказательства.

Удаление, случай 3: красный сын брата не с той стороны

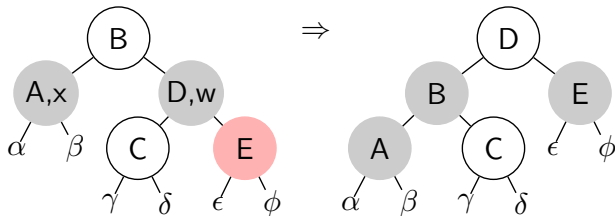
Случай 3. Брат w чёрный, его *левый* сын красный, а правый — чёрный. Перекрасить и повернуть **в самом** w .



Слева — вход в случай 3, справа — результат. **Черноту он не гасит:** лишний чёрный как висел на x , так и висит. Он переставляет брата так, чтобы красный сын оказался *справа* — и это ровно вход в случай 4. Братом x становится C , у него красный правый сын D .

Удаление, случай 4: чернота гасится, и цикл кончается

Случай 4. Брат w чёрный, его **правый** сын красный. Одна перекраска и Left-Rotate в отце:

$$\begin{aligned} color[w] &:= color[p[x]], & color[p[x]] &:= \text{BLACK}, & color[right[w]] &:= \text{BLACK}, \\ \text{Left-Rotate}(w, p[x]), & & x &:= \text{root} \end{aligned}$$


Слева — вход, справа — готовое дерево. **Лишняя чернота погашена:** её забрал красный сын E , которого перекрасили в чёрный. $x := \text{root}$ — это способ сказать «цикл кончился», а не подъём по дереву.

Вся процедура — одна колонка: четыре случая, два if

```
while x != root[T] and color[x] = BLACK
  if x = left[p[x]]
    w := right[p[x]]
    if color[w] = RED // 1
      color[w] := BLACK, color[p[x]] := RED
      Left-Rotate(w, p[x]); w := right[p[x]]
    if color[left[w]] = BLACK and color[right[w]] = BLACK
      color[w] := RED; x := p[x] // 2 -> выше
    else
      if color[right[w]] = BLACK
        color[left[w]] := BLACK // 3
        color[w] := RED
        Right-Rotate(left[w], w); w := right[p[x]]
      color[w] := color[p[x]] // 4
      color[p[x]] := BLACK
      color[right[w]] := BLACK
      Left-Rotate(w, p[x]); x := root[T]
  else // то же, с left <-> right
    color[x] := BLACK
```

Один листинг на четыре картинки предыдущих слайдов, и в нём нет строчки «на веру»: каждая отвечает за свой цвет на рисунке.

Отсюда единственное число, которое стоит запомнить: **поворотов за всё удаление не больше трёх** — по одному в 1, 3 и 4. Вверх идёт только случай 2, поэтому итераций $O(\log n)$, а поворотов — три.

Случай, записанный дважды, проверяется дважды: так удаление в КЧ и пишут неправильно — одну половину чинят, вторую забывают.

OS «Удаление. Алгоритм», с. 31–32 (оба слайда, левая половина); CLRS §13.4, RB-Delete-Fixup; в libstdc++ —

_Rb_tree_rebalance_for_erase (tree.cc), вызовы поворотов 374, 393, 400, 412, 430, 437.

Итог: за что платит AVL и за что — красно-чёрное

	AVL	КЧ
высота	$1,44 \lg n$	$2 \lg n$
вставка: поворотов	≤ 2	≤ 2
удаление: поворотов	$O(\log n)$	≤ 3
удаление: перекрасок	—	$O(\log n)$

Почему у AVL прочерк: у него *нет цветов*. В узле лежит баланс $-1/0/+1$; на удалении он правится вдоль всего пути — тоже $O(\log n)$, но это не перекраска, а те самые повороты строкой выше.

1,44 — теорема А Кнута (Е6, § 6.2.3, с. 460); $2 \lg n$ — Е7, шапка `lib/rbtree.c`; повороты — CLRS § 13.3–13.4, задача 13-3(a).

Вставка портит свойство 4 — «два красных подряд»: лечится в родителе или в деде.

Удаление портит свойство 5 — «одинаковое число чёрных на *всех* путях»: это про весь путь, поэтому черноту гасят вверх — и **поворотов за всё удаление не больше трёх**.

Почему везде стоит КЧ, а не AVL. AVL держит дерево ниже — КЧ вправе быть *на 39 % выше*, и поиск в нём чуть длиннее. Но AVL платит за это $O(\log n)$ **поворотами на каждом удалении**, а КЧ — не больше трёх. Удалений у словаря примерно столько же, сколько вставок, поэтому выгоднее КЧ: дерево чуть выше, зато правки дешёвые и с гарантией.

Отсюда КЧ в `std::map`, в ядре Linux и в `java.util.TreeMap`. AVL выигрывает там, где читают много, а меняют редко — и тогда, как мы увидим дальше, лучше вообще не дерево, а массив.

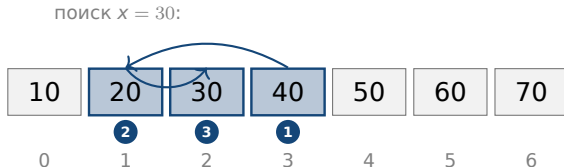
Где мы и куда идём

- Мы начали с **массива**: бинарный поиск, $\log_2 n$ сравнений, и упёрлись в то, что массив нельзя дёшево изменить.
- Ответом было **дерево**: БДП, AVL, красно-чёрное. Вставка и удаление стали дешёвыми, высота осталась логарифмической.
- За это дерево платит: узлы лежат в памяти вразброс, и **каждый шаг вниз — это обращение в память**, а не в кэш.

Дальше — третий ход: вернуть данные **обратно в массив**, сохранив дерево. Сначала поменяем *порядок хранения*, потом *размер узла*. Сравнений от этого не станет меньше — станет меньше промахов кэша.

Напоминание: поиск и его неявное дерево

```
1 size_t lo = 0, hi = n;           // тот же цикл, что до перерыва
2 while (lo < hi) {
3     size_t mid = lo + (hi - lo) / 2; // середина -- и есть узел дерева
4     if (a[mid] < x) lo = mid + 1; else hi = mid;
5 }
```



Индексы **прыгают**: середина, четверть, обратно. Соседей не читаем, пока отрезок не станет короче строки кэша — на это и жалуется следующий слайд.

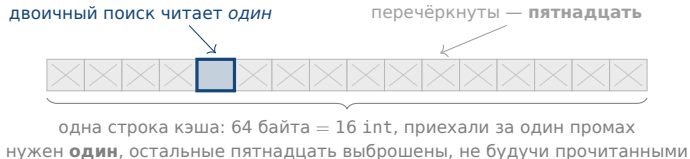
Но сравнения складываются в **дерево**, и оно уже здесь: корень — 3, дети — 1 и 5, листья — 0, 2, 4, 6. Мы его не строили. **Вся пара дальше — про то, как положить в память это дерево.**

Сначала — зачем: отсортированный массив плох для кэша

- **Временная локальность** — «only okay for the first dozen or so requests». Первые шаги *любого* поиска читают одни и те же ячейки: середину, потом одну из двух четвертей, потом одну из восьми — верхушку неявного дерева сравнений. Она и так в кэше от прошлых запросов, подкачивать её никто не просит. Глубже у каждого запроса свой путь.
- **Пространственная** — «only okay for the last 4–5 requests»: в строку кэша (64 байта = 16 int) соседи попадают только на последних шагах, когда отрезок уже короче строки.
- Между ними — середина поиска, где *каждый* шаг это новая строка кэша. Горячие и холодные элементы лежат вперемешку, «the opposite of what we want».

Дерево указателей из 2.1–2.4 плохо *по другой причине* — про неё в 2.6. Здесь мы меняем не структуру, а **порядок хранения**.

Строка кэша приезжает целиком — так используем её всю



- Ответ на это старше кэшей. Байер и Мак-Крейт, 1972: ключи держат не по одному, а **страницами устройства**, и «the pages of the index are organized in a special data-structure, so-called **B-trees**»; размер страницы — «a device dependent natural number».
- У них устройство — диск, страница — блок диска. У нас устройство — память, и правило то же: «for RAM, set B to be the cache line size so that we don't waste reads».
- То есть **узел** = **строка кэша**: $B = 16$ ключей по 4 байта.

Что это даёт: высота $\log_B n$ вместо $\log_2 n$



B ключей в узле делят диапазон на $B + 1$ частей: ветвление не 2, а 17. У Слотина это записано одной строкой — « $B \geq 2$ keys in each node, increasing branching factor and reducing tree height». При $B = 16$ высота падает в $\log_2 17 \approx \mathbf{4,09}$ раза:

n	2^{16}	2^{20}	2^{24}
уровней двоичного дерева, $\log_2 n$	16	20	24
уровней при $B = 16$, $\lceil \log_{17} n \rceil$	4	5	6

Цена: сравнений больше, а времени меньше

- Внутри узла ищут **перебором**: узел уже в кэше, и шестнадцать сравнений подряд дешевле одного похода в память. Цена названа прямо — «at the cost of having to perform B comparisons in each node», в среднем $B/2$ на узел.
- На $n = 2^{24}$: дерево указателей (узлы связаны указателями, каждый переход — обращение в память) — 24 сравнения и промах *на каждом* уровне; В-дерево — 6 уровней, ≈ 8 сравнений в узле, итого ≈ 48 сравнений и **6** походов в память. **Сравнений вдвое больше, походов вчетверо меньше.**
- Верхние уровни дерева оседают в кэше, так что 24 и 6 — верхние оценки: важно отношение. Свои числа — в замере №5.

Этот размен выбрали Linux, Rust и Abseil — к ним вернёмся в 2.7. **А сейчас — тот же приём при одном ключе в узле, $B = 1$:** то же дерево, разложенное по уровням, без указателей. Формально это уже не В-дерево (в определении выше $t \geq 2$) — но раскладка та же.

Мотивация: ключей больше, чем памяти

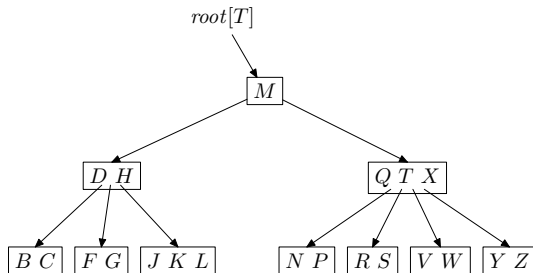
- Данные и ключи **не влезают** в оперативную память. Нужен простой механизм подтянуть ровно то, что нужно.
- Дереву это подходит: узлы легко загружать и выгружать. Но у двоичного дерева в узле *слишком мало* ключей — обращения к устройству получаются слишком частыми.
- Значит нужно сбалансированное дерево с **большой степенью ветвления**, в узлах которого хранятся тысячи ключей.

Это рассуждение 1972 года про диск. У нас то же самое, сменилось только устройство: один уровень дерева — одна строка кэша, а не блок диска. Отсюда и валюта всего блока: **высота = число походов в память.**

старая лекция 24.09.2010, к. «Мотивация»; алгоритмы и терминология — Кормен и др., «Алгоритмы: построение и анализ»,

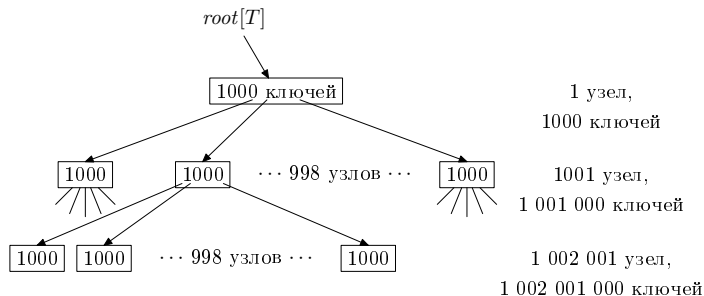
2-е изд., гл. 18 «В-деревья» (стр. 515–536 — из списка литературы той же лекции, по бумаге не сверено).

Пример B-дерева: $t = 2$, в узле от 1 до 3 ключей



Ключи в узле отсортированы; в поддереве между двумя соседними ключами — ключи строго между ними; все листья на одной глубине. **Поиск сравнивает с ключами узла слева направо и один раз спускается вниз:** ищем S — в корне $S > M$, правое поддерево QTX ; там $S > Q$, $S < T$ — второй указатель, лист RS ; в нём $S > R$, $S = S$ — нашли, узел RS , второй ключ.

Размер: три уровня — миллиард ключей



Указателей на один больше, чем ключей: 1000 ключей — 1001 ребёнок. Подписи справа считают *каждый свой* уровень: на третьем — 1001^2 узлов и 1 002 001 000 ключей. **Три обращения к устройству — на миллиард ключей.**

рисунок — старая лекция 24.09.2010, рис. 2; подписи справа пересчитаны с рисунка, их арифметика сверена.

Нестрогое понятие: В-дерево степени t

- **Идеально сбалансированно** — все листья на одной глубине.
- В узле **до** $2t - 1$ **ключей** и соответствующие им указатели на поддеревья.
- В любом узле, кроме корневого, **не менее** $t - 1$ **ключей**; в корневом — не менее одного.
- Ключи в узле отсортированы; в поддереве между двумя последовательными ключами — значения между ними.
- Ветвление — от t до $2t$, а не 2.

Нижняя граница $t - 1$ — не косметика: узел *никогда* не бывает пустым, и при удалении ключ всегда есть откуда взять. Верхняя $2t - 1$ — что полный узел всегда можно разрезать ровно пополам.

старая лекция 24.09.2010, к. «Нестрогое понятие В-дерева»; там же список литературы: Кормен и др., гл. 18; Кнут, т. 3,

§ 6.2.4 «Сильноветвящиеся деревья» (номера страниц — из списка 2010 г., по русскому изданию не сверялось).

Определение: что лежит в узле x

Узел x содержит:

1. $n[x]$ — число ключей в x ;
2. $n[x]$ ключей $key_1[x] \leq \dots \leq key_{n[x]}[x]$;
3. признак листа $leaf[x]$;
4. если x не лист — $n[x] + 1$ указатель $c_1[x], \dots, c_{n[x]+1}[x]$; у листа указателей нет.

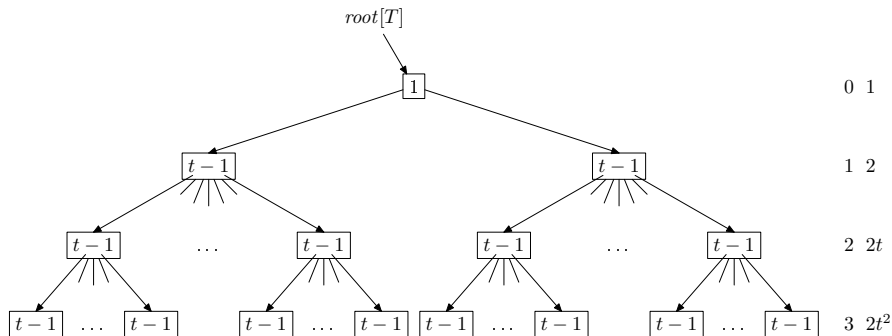
И два свойства дерева:

- все листья на одной высоте h ;
- t — минимальная степень: $t - 1 \leq n[x] \leq 2t - 1$ у всех x , кроме корня; в корне $1 \leq n \leq 2t - 1$.

Ключи-разделители задают диапазоны: если k_i — любой ключ из $c_i[x]$, то $k_1 \leq key_1[x] \leq k_2 \leq \dots \leq key_{n[x]}[x] \leq k_{n[x]+1}$.

Указателей **на один больше**, чем ключей — из-за этого ветвление $B + 1$, а не B .

Минимально заполненное дерево: худший случай по высоте



В корне 1 ключ, во всех остальных узлах — по $t-1$. Узлов на глубине $i \geq 1$ ровно $2t^{i-1}$ (справа на рисунке: 1, 2, $2t$, $2t^2$). Это **самое высокое** B-дерево из всех, в которых n ключей, — оно и даёт верхнюю оценку высоты.

Высота: $h \leq \log_t \frac{n+1}{2}$ — доказательство в строку

Утверждение. В В-дереве с $n \geq 1$ ключами и минимальной степенью t высота $h \leq \log_t \frac{n+1}{2}$.

Считаем ключи минимально заполненного дерева:

$$n \geq 1 + (t-1) \sum_{i=1}^h 2t^{i-1} = 1 + 2(t-1) \frac{t^h - 1}{t-1} = 2t^h - 1 \quad \Rightarrow \quad t^h \leq \frac{n+1}{2}.$$

Сумма геометрическая, основание — t . Дырки в доказательстве нет: левая часть и есть картинка предыдущего слайда.

При $t = 8$ (15 ключей в полном узле — почти строка кэша) и $n = 10^9$ получается $h \leq \log_8(5 \cdot 10^8) \approx 9,6$, то есть ≤ 10 **уровней против** ≈ 30 у двоичного дерева того же размера.

старая лекция 24.09.2010, к. «Минимальная высота В-деревя»; Кормен и др., 2-е изд., лемма 18.1. Число уровней при

$n = 10^9$ — арифметика по формуле слайда, не замер; отдельной строки в numbers.md нет.

Операции: у дерева есть только два способа что-нибудь узнать

- `Disk-Read(x)` — прочитать узел из внешней памяти;
- `Disk-Write(x)` — записать узел обратно.
- Конкретная реализация подразумевает LRU-буфер узлов некоторого размера: часто используемые остаются в памяти.
- В алгоритмах корень считается всегда висящим в памяти. На практике это необязательно: при LRU-буферизации он и так из памяти не уходит — это самый горячий узел дерева.

Отсюда правило чтения листингов ниже: **каждый** `Disk-Read` — это уровень дерева, и ни один `Disk-Read` не случается *внутри* узла. В нашем переводе `Disk-Read` = промах кэша.

Создание: В-дерево начинается с одного пустого листа

```
1 proc BTree-Create(T):           // дерево из одного пустого листа
2     x <- Allocate-Node()
3     leaf[x] <- true
4     n[x]    <- 0
5     Disk-Write(x)               // пустой узел уезжает на устройство
6     root[T] <- x
```

Отдельной «инициализации» нет: пустое В-дерево — это один лист с нулём ключей. Всё остальное сделает вставка, и она же вырастит новый корень, когда старому станет тесно.

старая лекция 24.09.2010, к. «Создание В-дерева»; алгоритм — Кормен и др., 2-е изд., B-TREE-CREATE (гл. 18.1). Псевдокод пересказан, а не цитирован.

Поиск: перебор по узлу, один Disk-Read — на уровень

```
1 proc BTree-Search(x, k):
2     i <- 1
3     while i <= n[x] and k > key_i[x]:    // перебор внутри узла:
4         i <- i + 1                        // до n[x] сравнений
5     if i <= n[x] and k = key_i[x]:
6         return (x, i)                    // нашли: узел и номер ключа в нём
7     if leaf[x]:
8         return nil                       // лист, ключа нет — искать негде
9     Disk-Read(c_i[x])                    // ОДИН поход в память на уровень
10    return BTree-Search(c_i[x], k)
```

Высота h — значит $h + 1$ обращений к устройству и $\leq n[x]$ сравнений в каждом узле: **в среднем $B/2$ на узел**. Внутри узла ищут перебором, а не делением пополам.

старая лекция 24.09.2010, к. «Поиск в B-дереве»; алгоритм — Кормен и др., 2-е изд., B-TREE-SEARCH (гл. 18.1). Про $B/2$ —

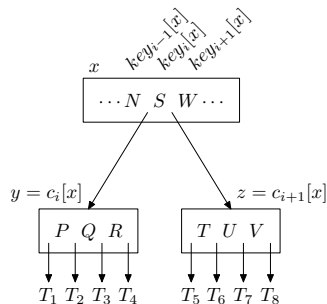
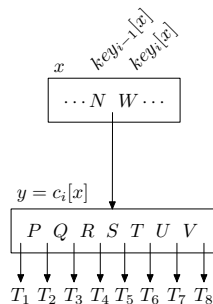
S14 в numbers.md.

Вставка: лист ищут вниз, узлы делят на пути

- Ищется лист, куда подходит новый ключ.
- Нельзя просто создать новый лист: **в листе обязано быть не менее $t - 1$ ключей.**
- Полный узел ($2t - 1$ ключей) делят пополам **по медиане**: медиана поднимается в родителя, правая половинка становится новым узлом.
- Чтобы уложиться в **один проход**, при спуске делят все полные узлы — даже те, до которых вставка в итоге не дойдёт.

Пункт 4 — не расточительность, а единственная причина, по которой вставка обходится без подъёма обратно вверх: спускаясь, мы всегда держим **незаполненного** родителя, и делить ребёнка есть куда.

Разбиение узла при $t = 4$: полный узел режут по медиане



Слева — $y = c_i[x]$ полон: $2t - 1 = 7$ ключей, восемь детей. Справа — y оставил себе первые $t - 1 = 3$, новые z — последние три, медиана $key_t = S$ **поднимается в родителя**, детей y у становится t , а z забирает остальные.

рисунки — старая лекция 24.09.2010, рис. 4 и 5 (разбиение узла); сверены с собранным b-tree.pdf. Тот же разбор —

Разбиение: три массива сдвигаются, три узла записываются

```
1  proc BTree-Split-Child(x, i, y):          // y = c_i[x], и он полон: n[y] = 2t-1
2      z <- Allocate-Node()
3      leaf[z] <- leaf[y]                    // z наследует я" лист"
4      n[z]    <- t - 1
5      for j <- 1 to t-1:  key_j[z]  <- key_{j+t}[y]  // правая половина
6      if not leaf[y]:
7          for j <- 1 to t:  c_j[z]  <- c_{j+t}[y]    // и её дети
8      n[y] <- t - 1
9      for j <- n[x] downto i+1: c_{j+1}[x] <- c_j[x]  // в родителе
10     c_{i+1}[x] <- z                                // раздвигаем
11     for j <- n[x] downto i:  key_{j+1}[x] <- key_j[x] // массивы
12     key_i[x]  <- key_t[y]                          // медиана поднимается в x
13     n[x] <- n[x] + 1
14     Disk-Write(y); Disk-Write(z); Disk-Write(x)    // три узла на диск
```

Ни одного обхода дерева, ни одного перехода: **три узла и два сдвига в массивах.**

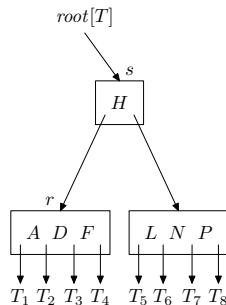
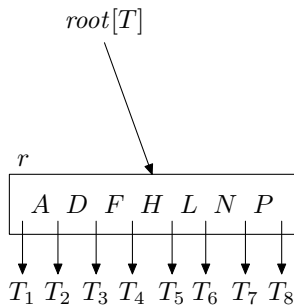
старая лекция 24.09.2010, листинг BTree-Split-Child; он же — Кормен и др., 2-е изд., B-TREE-SPLIT-CHILD (гл. 18.2).

Вставка: единственный случай, когда дерево растёт вверх

```
1  proc BTree-Insert(T, k):
2      r <- root[T]
3      if n[r] = 2t - 1:                // корень полон – делать новый корень
4          s <- Allocate-Node()
5          leaf[s] <- false
6          n[s] <- 0
7          c_1[s] <- r
8          BTree-Split-Child(s, 1, r)   // медиана r – единственный ключ s
9          BTree-Insert-Nonfull(s, k)
10     else
11         BTree-Insert-Nonfull(r, k)    // обычно – сразу вниз
```

Высота В-дерева растёт **только здесь** — и только когда полон корень. Значит она растёт равномерно: ни одна ветвь не убегает вперёд другие, и «идеально сбалансированно» из определения — не обещание, а следствие этой единственной строки.

Разбиение корня при $t = 4$: дерево выросло на один уровень



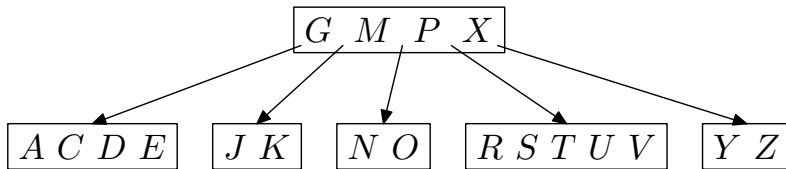
Слева — полон корень r : $ADFHLNP$. Справа — новый корень s из одного ключа H , под ним два узла по $t - 1 = 3$ ключа. Высота $+1$, и **это единственный способ, каким В-дерево растёт**.

Вставка в незаполненный узел: полные узлы разбиваем на спуске

```
1 proc BTree-Insert-Nonfull(x, k):          // на входе всегда  $n[x] \leq 2t-2$ 
2     i ← n[x]
3     if leaf[x]:
4         while i ≥ 1 and k < key_i[x]: // сдвигаем хвост вправо
5             key_{i+1}[x] ← key_i[x]; i ← i - 1
6             key_{i+1}[x] ← k; n[x] ← n[x] + 1; Disk-Write(x)
7     else
8         while i ≥ 1 and k < key_i[x]: i ← i - 1      // выбор ветви
9         i ← i + 1; Disk-Read(c_i[x])
10        if n[c_i[x]] = 2t - 1:                        // сын полон —
11            BTree-Split-Child(x, i, c_i[x])          // делим ДО спуска
12            if k > key_i[x]: i ← i + 1                // правая половина
13            BTree-Insert-Nonfull(c_i[x], k)
```

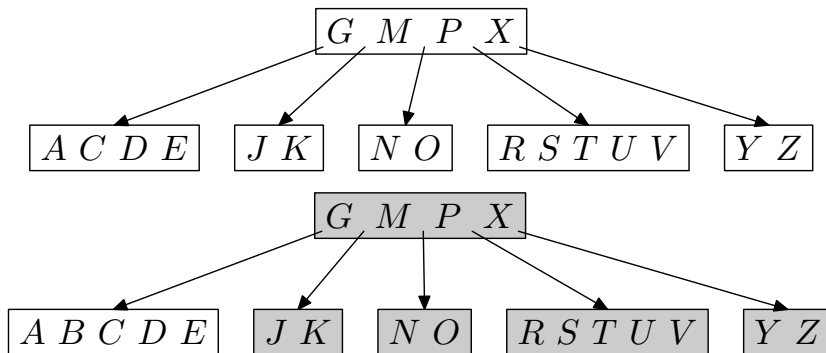
Инвариант вызова — $n[x] \leq 2t - 2$: в узел всегда есть что положить.
Отсюда — ни одного возврата вверх по дереву.

Вставка по шагам, $t = 3$: узел держит от 2 до 5 ключей



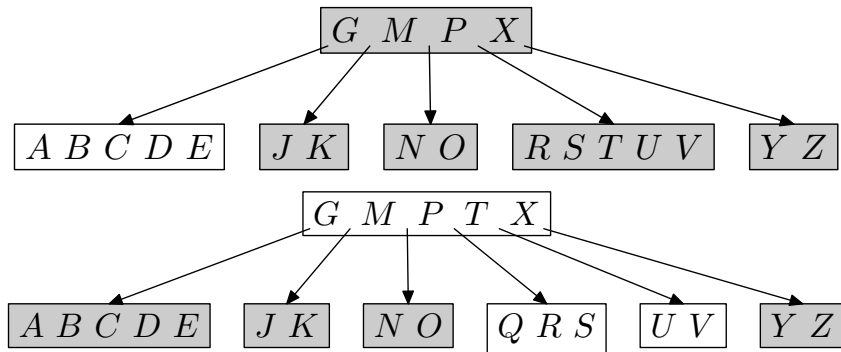
Полный узел — 5 ключей ($RSTUV$), все остальные — неполные. Вставляем по очереди четыре ключа: B, Q, L, F . На каждом следующем слайде сверху — дерево «до», снизу — «после».

Вставка B : лист не полон — просто вставка



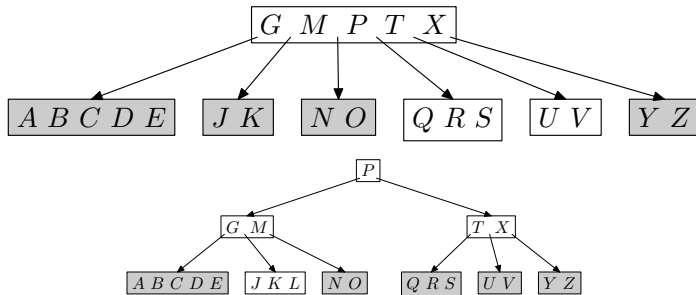
$B < G \rightarrow$ первый сын, лист $ACDE$. Ключей было четыре — это $2t - 2$, инвариант выполнен, делить нечего: сдвигаем хвост и пишем узел. Стал полным — но **делят узлы до спуска в них**, а не после вставки.

Вставка Q: первое разбиение — полон лист



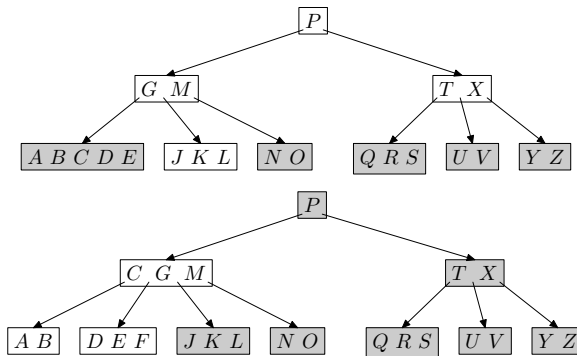
$Q > P$, $Q < X \rightarrow$ четвёртый сын, $RSTUV$ — **он полон**, поэтому до спуска делим его: медиана T поднимается в корень, UV — новый брат. Дальше спуск туда, где ключ: $Q < T \rightarrow$ левая половина, RS , и в неё вставка $\rightarrow QRS$.

Вставка L : полон корень — единственный рост вверх



Корень $GMPTX$ — **полон** (5 ключей), поэтому по правилу «делить заранее» его делят, ещё не зная, куда пойдём: медиана P поднимается — **появился новый корень**, высота $+1$. Дальше обычный спуск: $L < P \rightarrow$ левый сын GM , там L между G и $M \rightarrow$ лист $JK \rightarrow JKL$.

Вставка F : разбиение листа, у которого внутренний родитель



$F < P \rightarrow GM$ (не полон, не делим); $F < G \rightarrow$ лист $ABCDE$ — **он полон**.
Делим: медиана C поднимается в $GM \rightarrow CGM$, правая половина DE — новый брат. $F > C \rightarrow$ спускаемся в DE и вставляем $\rightarrow DEF$.

Итог вставки: один проход вниз, высота растёт только сверху

- B — вставка в неполный лист: сдвиг хвоста, один Disk-Write.
- Q — полон лист: делим до спуска, медиана в родителя.
- L — полон корень: новый корень, **высота +1**.
- F — полон лист под внутренним узлом: растёт родитель — и ключом, и указателем.

Стоимость вставки — спуск до листа плюс записи затронутых узлов: тот же порядок, что у поиска. При $B = 16$ это **6 уровней вместо 24** — та же высота, что на слайде «Что это даёт» (Y1): широкие узлы дёшевы не только в поиске.

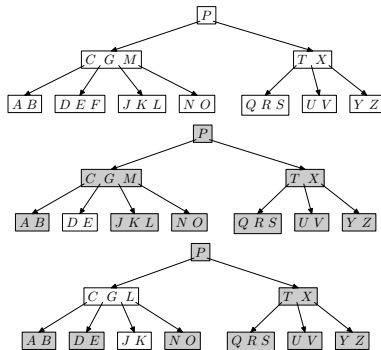
пример — старая лекция 24.09.2010, рис. 8–12; 6 уровней против 24 — арифметика по Y1 (numbers.md), не замер.

Удаление: вместо разбиения — слияние и перетекание

та же схема «заранее», только узлы не делят, а **наполняют**, не доходя до них:

1. k лежит в x , и x — лист: просто удалить из x .
2. k лежит в x , x — внутренний: взять предшественника или наследника k' и **заменить им** k , удалив k' ниже.
3. k во внутреннем узле x нет: идём в $c_i[x]$, но только если в нём уже t ключей. Иначе **довести его до t** :
 - сосед содержит $\geq t$ — *перетекание*: разделитель из x опускается, ключ соседа поднимается;
 - оба соседа по $t - 1$ — *слияние*: сосед, разделитель и $c_i[x]$ в один узел, разделитель — его медиана.

Удаление F и M : лист и внутренний узел

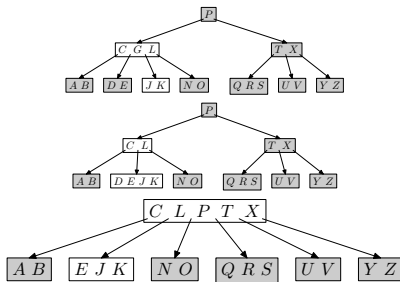


Сверху вниз: до, после F , после M . F — случай 1: в листе DEF было три ключа, стало два $= t - 1$, узел легален, больше ничего не делаем. M — случай 2: M во внутреннем узле; предшественник — L из JKL : удаляем L оттуда и заменяем им M : $CGM \rightarrow CGL$, $JKL \rightarrow JK$.

рисунки — старая лекция 24.09.2010, рис. 13, 14, 15 ($t = 3$); переходы сверены по рисункам: $DEF \rightarrow DE$; $CGM \rightarrow CGL$,

$JKL \rightarrow JK$.

Удаление G и D : два слияния, высота падает

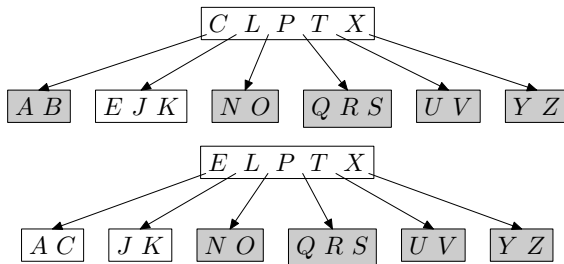


G — не в листе, а в узле CGL : удаляемый ключ — сам разделитель между DE и JK , оба листа по $t - 1$ — **слияние** $DE + G + JK \rightarrow DEJK$, $CGL \rightarrow CL$. D — путь через CL (по минимуму, соседи тоже), разделителем оказался **корень** P : $CL + P + TX$ — новый корень, **высота** -1 ; затем D уходит: $DEJK \rightarrow EJK$.

рисунки — старая лекция 24.09.2010, рис. 15, 16, 17 ($t = 3$); переходы сверены: $DE + G + JK \rightarrow DEJK$; на рис. 17 корень — уже

$CLPTX$, шесть листьев.

Удаление B : перетекание ключа через родителя



B — в листе AB , это $t - 1$, своего ключа не хватит. Правый сосед EJK содержит $\geq t$ — **перетекание**: разделитель C опускается в левый лист, минимальный ключ соседа E поднимается в корень.

$AB \rightarrow ABC$, $EJK \rightarrow JK$, корень $CLPTX \rightarrow ELPTX$. И только потом B удаляется из листа: $ABC \rightarrow AC$. **Ни уровня, ни лишнего ключа** — узлы просто переложились.

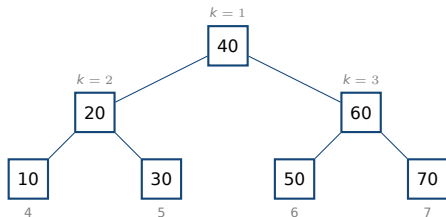
В-дерево: что взяли и что осталось незакрытым

- Широкий узел — не приём, а **следствие**: высота = число обращений к устройству, а устройство меняется (диск → строка кэша).
- Ветвление $B + 1$; внутри узла — **перебор**, $B/2$ сравнений (S14): сравнений больше, походов в память вчетверо меньше (Y2).
- Вставка и удаление — **один проход вниз**: полные узлы делят заранее, пустеющие — наполняют заранее.
- Высота лежит между $\log_{2t}(n + 1) - 1$ и $\log_t \frac{n+1}{2}$, и меняется только вместе с корнем.

Незакрытое: у узла **17 указателей**. С ними строка кэша перестает быть узлом: 16 ключей по 4 байта — 64 байта, и ещё 17 указателей по 8 — итого 200 байт, а строка — 64. Дальше — как указатели выкинуть совсем.

Раскладка Эйтингера: то же дерево, разложенное по уровням

Михаэль Эйтингер, XVI век, *ahnentafel* — таблица предков: субъект — номер 1, отец человека k — номер $2k$, мать — $2k + 1$. Та же нумерация, что в двоичной куче и в дереве отрезков.



отсортированный: 10 20 30 40 50 60 70

Эйтингер $t[1..7]$:

40 20 60 10 30 50 70

указателей нет — адрес считается

Это **то же самое дерево**, которое мы рисовали полчаса назад. Узлы просто разложены в массив по уровням, и left/right больше не нужны: сын лежит по вычисляемому адресу.

Эйтингер — та же раскладка 2.5a при $B = 1$: узел на один ключ, адрес ребёнка считается, указателей нет вовсе.

Построение за $O(n)$, поиск в три строки

```
1 // build: обход in-order кладёт ключи по своим местам за  $O(n)$ 
2 void build(int k = 1) {
3     if (k <= n) { build(2 * k); t[k] = a[i++]; build(2 * k + 1); }
4 }
5 // search: ни одной ветки и ни одного указателя
6 int k = 1;
7 while (k <= n) k = 2 * k + (t[k] < x);
8 k >>= __builtin_ffs(~k);      // и вот здесь ответ
```

Обход build — тот же in-order, что в 2.1. Цикл поиска — один lea-подобный шаг на уровень; **перехода в нём нет.**

Последняя строка — не фокус, а разбор пути. Каждый шаг дописывает к k бит: **0** — свернули влево ($t[k] \geq x$, узел годится в ответ), **1** — вправо. В конце k — единица и весь путь битами. Ответ — *последний левый поворот*: сбрасываем хвост из единиц и ещё один бит. Сколько — позиция младшего нуля в k , её и даёт `__builtin_ffs(~k)`.

$n = 15$, $x = 35$: $k = 10011_2$, сбрасываем 3 бита, $k = 2$, ответ $t[2] = 40$.

свой код, не цитированный (правило рецепта); приём — SL, стр. 34–39; разбор последней строки —

Замер №4: Эйтцингер против библиотеки

демо

n	2^{16}	2^{20}	2^{24}	2^{27}
Эйтцингер, нс на запрос	27,5	39,3	161,9	255,8
безветвенный, нс	43,7	102,2	762,1	—
<code>std::lower_bound</code> , нс	51,0	91,7	430,5	955,9
std / Эйтцингер	1,85	2,33	2,66	3,74

Быстрее и библиотеки, и безветвенного варианта — **на всех** измеренных n , и разрыв растёт с n . Предвыборка здесь окупается: за одну строку кэша приезжают все четыре внука, поэтому хватает одного `__builtin_prefetch(&t[k * 16])` при `alignas(64)`.

Сравнений при этом ровно столько же, сколько было.

наши числа — `demo/bench-binsearch/results.md`, «Эксперимент 4», строка B6; стенд M1. Приём — SL, стр. 42–46. Заявка

автора «Eytzinger: 2x on large arrays» (S6) — по его данным, на его железе (Zen 2 @ 2 ГГц, DDR4-2667).

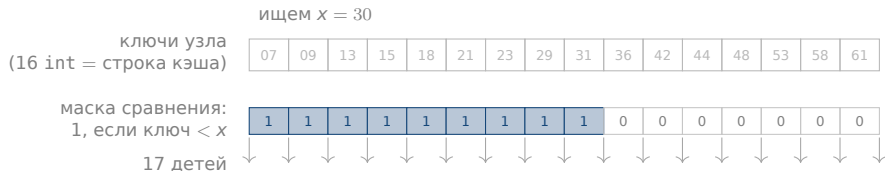
Отдельно — про метод: измерение на новом железе перевернуло вывод

Khuong и Morin (2015) в аннотации пишут, что их вывод *«goes counter to earlier experimental work by Brodal, Fagerberg, and Jacob (SODA 2003), who concluded that both the B-tree and van Emde Boas layouts were faster than the Eytzinger layout for large values of n »*.

- Обе работы — честные эксперименты. Обе правы для своего железа.
- За двенадцать лет изменилось соотношение цены сравнения и цены промаха кэша — и порядок структур перевернулся.
- Значит, у измеренного результата есть **срок годности**, и его надо называть вместе с числом.

То же самое показал день 1 на пороге отсечения introsort.

Узел на 16 ключей: 17 детей без единого указателя



`_mm256_cmpgt_epi32` сравнивает **восемь** int за одну инструкцию, значит на шестнадцать ключей нужно два таких сравнения. Вторая строка — их результат. **Сумма единиц = 9 — это и есть номер сына**, в которого спускаемся. Сравнений в узле шестнадцать, а не четыре, — зато ни одного перехода и ни одного указателя: адрес сына *считается*, $k \cdot (B + 1) + i + 1$ (S13).

При $B = 1$ та же формула даёт $2k + 1$ и $2k + 2$ — это **раскладка Эйтцингера из 2.5** с одним ключом в узле. S-tree — она же при $B = 16$.

S-tree: узел — строка кэша, сравнение — одна инструкция

- Два куска складываются в один: узел размером со строку кэша — из 2.5a, адрес сына из номера отца — отсюда. «We can make static B-trees implicit by generalizing the Eytzinger numeration»: корень 0, дети узла k — $k \cdot (B + 1) + i + 1$, $i \in [0, B]$.
- При $B = 1$ это $2k + 1$ и $2k + 2$ — раскладка, которую мы только что написали. **S-tree — она же при $B = 16$.**
- Перебор внутри узла — « $B/2$ comparisons on average, unpredictable branching»: та самая непредсказуемая ветка из 1.10, и убирают её так же — маской сравнений без переходов.
- Дальше SIMD: одной инструкцией сравнить ключ со всем блоком. AVX2 на Zen 2 — $256/32 = 8$ ключей; на нашем M1 есть AVX-512, это 16 — **весь узел за одну инструкцию.**

Сколько это стоит — и чего мы не проверяли

Итог доклада: **S-tree** — **5-8x**, **S+ tree** — **7-15x** относительно `std::lower_bound`; в аннотации доклада — «up to ~**15x** faster than `std::lower_bound`».

Оговорки с его же слайдов: 32-битные целые, случайные ключи и запросы, оптимизация на *пропускную способность*, и это «not a drop-in `std::lower_bound` replacement». По данным автора, на его железе (Zen 2 @ 2 ГГц, DDR4-2667).

Своих чисел для S-tree у нас нет и к сегодняшнему дню не будет: измерены наивный, безветвенный, с предвыборкой и Эйтцингер. И это не придирка — безветвенный вариант у автора даёт «3x on small arrays», а у нас на 2^{16} вышло 1,16x. Чужой множитель на свою машину не переносится.

SL, стр. 3 и 4 (S8), аннотация — youtube.com/watch?v=1RIPMQRBWk (S9), проверено 16.09.2026; «3x on small arrays» — S3;

наши 1,16 — B4.

Замер №5: одна задача, три структуры

демо

Один и тот же `lower_bound` в трёх структурах: `std::map`, отсортированный `std::vector` и наш Эйтцингер. **Нс на запрос**, на 2^{24} — те же промахи кэша на запрос и **время построения**.

Ожидание вслух до запуска: «у дерева и у массива одинаковая асимптотика $O(\log n)$ — значит, они должны идти вровень».

нс на запрос	<code>std::map</code>	вектор	Эйтцингер
$n = 2^{10}$	23,5	25,7	17,6
$n = 2^{16}$	95,7	52,8	27,7
$n = 2^{20}$	308,6	91,7	39,4
$n = 2^{24}$	1 443,0	424,9	158,3
<i>на том же 2^{24}</i>			
промахов кэша на запрос	22,4	80,6	8,3
построение, нс на ключ	103,1	69,9	1,7

Полные ряды — от 2^{10} до 2^{24} , 15 размеров; счётчики кэша и построение сняты на четырёх размерах, по одной точке на режим (в кэше / на границе кэша / в памяти).

Почему дерево указателей проигрывает — словами Rust

Документация `std::collections::BTreeMap`, дословно:

*«In theory, a binary search tree (BST) is the optimal choice for a sorted map... However, in practice the way this is done is very inefficient for modern computer architectures. In particular, every element is stored in its own individually heap-allocated node. This means that every single insertion triggers a heap-allocation, and **every comparison is a potential cache-miss** due to the indirection.»*

Дерево указателей — узлы связаны указателями: шаг вниз — отдельное обращение в память, адрес следующего узла неизвестен, пока не прочитан текущий. Против него — массив, лежащий *подряд*.

Отсюда **один промах кэша на уровень** — и формулировка, ради которой всё: $O(\log n)$ **прячет константу размером с обращение в памяти**.

Чем заменяют красно-чёрные деревья: ядро и Rust

1. **Linux 6.1 (2022) — Maple Tree.** Это B-дерево, у которого в узле не отдельные ключи, а *отрезки*: ядру нужно «какой участок памяти покрывает этот адрес», а не «есть ли такой ключ». Плюс **RCU**: читатели ходят по дереву *без блокировки*, пока писатель строит новую версию узла. Узел подобран под кэш — та же арифметика, что в 2.5a. Словами авторов: «an RCU-safe range based B-tree designed to use modern processor cache efficiently». Документация ядра: «The most important usage of the Maple Tree is the tracking of the **virtual memory areas**» — то есть VMA, которые в списке применений КЧ-деревьев стоят прямым текстом, переехали на B-дерево.
2. **Rust** — красно-чёрного дерева в стандартной библиотеке *нет вообще*. Есть BTreeMap, и внутри узла — «our implementation simply performs **naive linear search**»: те самые «сравнений больше, походов в память меньше» из 2.5a, выбранные сознательно и записанные в документацию.

[kernelnewbies.org/Linux_6.1 §1.7](http://kernelnewbies.org/Linux_6.1%20%26%201.7) (E9); kernel.org/doc/html/latest/core-api/maple_tree.html (E10); документация

BTreeMap (E11).

Тот же размен в C++: `absl::btree_map`

Google Abseil даёт `btree_map` и `btree_set` как замену `std::map` / `std::set`. Причина названа в самом заголовке:

*«Unlike `std::map` ...commonly implemented using **red-black tree nodes**, B-tree maps use more generic B-tree nodes able to hold **multiple values per node**. Holding multiple values per node often makes B-tree maps perform better ...because multiple entries can be checked within the **same cache hit**.»*

Это ровно наш слайд про цену: сравнений в узле больше, походов в память меньше. Руководство Abseil: «lower memory overhead in most cases», «generally more cache friendly (and hence faster)».

А теперь цена, и она знакомая. Тот же заголовок, абзацем ниже:

*«Importantly, insertions and deletions may **invalidate outstanding iterators, pointers, and references** to elements.»*

Вспомните слайд про `std::map`: `std::map` *обязан* сохранять все итераторы, кроме одного, и поэтому `libstdc++` перевязывает узел, а не копирует ключ. **Abseil это обещание отдаёт** — и получает взамен много ключей в узле. Узел на один элемент — это и была плата за обещание.

`absl/container/btree_map.h`, шапка файла (прочитано 19.09.2026); руководство — abseil.io/docs/cpp/guides/container;

гарантия `std::map` — слайд про `std::map`.

И — что осталось: не «устарели», а сменилась цена памяти

Документация ядра перечисляет, где красно-чёрные деревья в Linux живут до сих пор: планировщики ввода-вывода deadline и CFQ, драйвер CD/DVD, **таймеры высокого разрешения**, каталоги ext3, epoll, криптографические ключи, планировщик HTB.

А `java.util.TreeMap` — это по-прежнему «*A Red-Black tree based NavigableMap implementation... Algorithms are adaptations of those in Cormen, Leiserson, and Rivest's Introduction to Algorithms.*»

— тот самый CLRS из 2.4, названный в javadoc.

Верная мораль: сменилась не асимптотика, а стоимость обращения к памяти. Структура, у которой узел — одна строка кэша, выигрывает у структуры, у которой узел — один указатель.

Стена: почему быстрее $\log_2 n$ сравнений нельзя

- У задачи `lower_bound` на массиве из n ключей ровно $n + 1$ возможных ответов — индексы от 0 до n .
- Алгоритм, который умеет *только сравнивать*, — это двоичное дерево решений. У Кнута дословно: «*any algorithm for searching an ordered table of length N by means of comparisons can be represented as an N -node binary tree*».
- В дереве должно быть не меньше $n + 1$ листьев, а у двоичного дерева высоты h их не больше 2^h . Значит $h \geq \lceil \log_2(n + 1) \rceil$.
- Бинарный поиск ровно столько и делает: «*Algorithm B is optimum in this sense, for all N* ».

Ровная рифма с днём 1: там было $n!$ листьев и $\Omega(n \log n)$.

Как её ломают: перестают сравнивать

Кнут, § 6.4, первая страница — три способа искать:

«*So far we have considered search methods based on **comparing** the given argument K to the keys in the table, or using **its digits** to govern a branching process. A third possibility is to avoid all this rummaging around by doing some arithmetical **calculation on K** , computing a function $f(K)$ that is the location of K .*»

- **Сравнения** — весь сегодняшний день до этого слайда. Стена $\Omega(\log n)$.
- **Цифры — цифровой поиск**: бор (trie). На каждом шаге смотрят не «больше/меньше», а очередной разряд ключа, и он сам выбирает ветвь. У Кнута это отдельный параграф — § 6.3 «Digital Searching», следующий сразу за нашим 6.2. Рифма с днём 1: там по разрядам *сортировали*, здесь по ним *ищут*.
- **Вычислить адрес** — хеш-таблица. Следующий пункт.

Хеш-таблица: идея за одну минуту

- Дерево *сравнивает* ключ с другими ключами. Хеш-таблица **вычисляет по ключу адрес** и идёт прямо туда.
- $h(\text{ключ}) \rightarrow$ номер ячейки в массиве. Один вызов функции, одно обращение в память — и не важно, сколько всего ключей.
- Плата: два разных ключа могут дать один номер — **коллизия**. Разрешают либо цепочкой (список в ячейке), либо открытой адресацией (ищут соседнюю свободную).
- И вторая плата, важная для сегодняшнего сюжета: **порядок теряется**. Соседние по значению ключи лежат где попало, поэтому «следующий за x » хеш-таблица не отвечает.

`std::unordered_map`, `java.util.HashMap`, `dict` в Python — это всё хеш-таблицы. Слово «unordered» в имени — ровно про потерянный порядок.

SwissTable — что это и откуда

- Хеш-таблица из библиотеки **Abseil** (Google, открыта в 2017). Сейчас это основная хеш-таблица внутри Google, а её устройство повторили `hashbrown` в Rust (стандартный `HashMap` с 2018) и `flat_hash_map` в других библиотеках.
- **Плоская**: пары ключ-значение лежат прямо в массиве, без узлов и без указателей. Открытая адресация вместо цепочек.
- Рядом с данными — **массив управляющих байтов**, по байту на слот: пусто, удалено, или 7 бит хеша.
- Поиск сравнивает **сразу 16 управляющих байтов** одной векторной инструкцией и получает маску кандидатов. В сами ключи заглядывает только по совпавшим битам маски.

То есть выигрыш не в формуле хеша, а в том, что таблица *укладывается в кэш* и один шаг проверяет шестнадцать слотов.

Почему `std::unordered_map` медленный — и что вместо

- Стандарт требует стабильности ссылок и семантики корзины. Значит — узлы в куче, **одна аллокация на элемент** и указатель на каждом шаге цепочки. Та же болезнь, что у `std::map` в 2.6.
- SwissTable: **плоская** таблица, открытая адресация и **массив управляющих байтов** рядом с данными.
- Хеш делится на две части. H_1 — «a 57 bit hash value, used to identify the element index within the table itself»; H_2 — «the remaining 7 bits of the hash value, used to store metadata for this element».
- Цена: «This metadata adds **one byte of overhead** for every entry in the table».

Шестнадцать кандидатов — за несколько инструкций

```
1 Mask Match(h2_t hash) const {  
2     auto match = _mm_set1_epi8(hash);  
3     return Mask(_mm_movemask_epi8(_mm_cmpeq_epi8(match, metadata)));  
4 }
```

Группа — 16 соседних слотов и отдельный массив из **16 управляющих байтов**, по байту на слот. Ищем ключ с $H_2 = 2A$:

управляющие байты	2A	7F	11	2A	03	5E	2A	C1	40	9B	17	2A	68	D4	05	33
маска Match	1	0	0	1	0	0	1	0	0	0	0	1	0	0	0	0
слоты																

Одно `_mm_cmpeq_epi8` сравнивает **все 16 байтов сразу**, `_mm_movemask_epi8` собирает результат в 16-битную маску. Читать сами ключи нужно только в четырёх слотах из шестнадцати — «we can **winnow 16 candidates** down to only those with a matching H2 hash in only a few instructions». Коэффициент заполнения — из комментария в коде: «We use **7/8th** as maximum load factor. For 16-wide groups, that gives an average of **two empty slots per group**.»

Документация и код разошлись — проверено

16.09.2026

```
984 inline size_t H1(size_t hash) { return hash; }
```

```
989 inline h2_t H2(size_t hash) { return hash >> (sizeof(size_t) * 8 - 7); }
```

В проектной записке H_1 — «а 57 bit hash value». В коде H_1 — это весь хеш, а H_2 берётся из **старших** семи бит, а не из «оставшихся». Записку писали, чтобы объяснить замысел; код с тех пор поехал. Ни то ни другое не ошибка — ошибкой было бы рассказать вам замысел как устройство.

Читать надо код. Записку — как объяснение, зачем он такой.

absl/container/internal/raw_hash_set.h (ветка master, прочитано 17.09.2026), строки 984 и 989 — код C8; записка — E14.

Куда это разошлось — и два числа из одного источника

- **Rust** перевёл HashMap на порт SwissTable (hashbrown). README: «Around **2x faster** than the previous standard library HashMap. Lower memory usage: only **1 byte** of overhead per entry instead of **8.**»
- **Go 1.24** (26.02.2025) перевёл встроенный map: «In microbenchmarks, map operations are up to **60%** faster than in Go 1.23... in full application benchmarks, we found a geometric mean CPU time improvement of around **1,5%.**»

Тот же источник, тот же абзац, две цифры: **60 %** и **1,5 %**. И разница между ними — это вся разница между микробенчмарком и приложением.

Когда вам называют ускорение, первый вопрос — *чего именно*.

Замер №6: и столбец, которого у хеша нет

демо

Точечный поиск по существующему ключу: `std::unordered_map`, наша плоская таблица (открытая адресация, управляющие байты — упрощённая SwissTable), `std::map` и отсортированный вектор. **Нс на поиск**, стенд M1 + perf.

нс на поиск	<code>unordered_map</code>	плоская	<code>std::map</code>	вектор
$n = 2^{10}$	9,4	7,5	29,7	34,1
$n = 2^{16}$	30,3	11,2	103,7	69,0
$n = 2^{20}$	167,0	19,6	387,8	127,3
$n = 2^{24}$	473,0	159,8	1 859,7	764,8

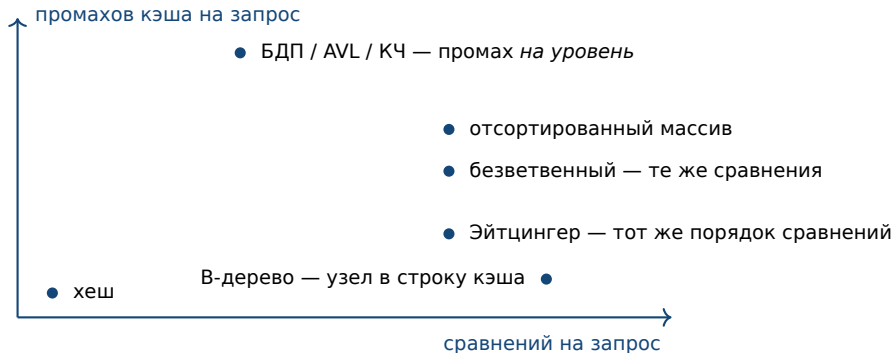
	точный поиск	<code>lower_bound</code>
отсортированный массив, Эйтцингер	да	да
<code>std::map</code> , AVL, КЧ, В-дерево	да	да
хеш-таблица	да, быстрее всех	нет. Никогда

Хеш не отвечает на «первый $\geq x$ » *ни за какое время*: он вычисляет адрес, порядок теряется.

Одной строкой каждое

- **ART** (Leis, Kemper, Neumann, ICDE 2013) — цифровое дерево с узлами переменного размера (4/16/48/256), Node16 ищется одной инструкцией SSE. Умеет и точечный запрос, и диапазонный. Прямой родственник поразрядной сортировки из дня 1; используется в DuckDB.
- **Интерполяционный поиск**: RocksDB включил его для индексных блоков SST — 335 749 → 366 598 операций/с, +9,2%. Оговорка *самих авторов*: синтетический тест на равномерных ключах.
- **Learned indexes** (Kraska et al., SIGMOD 2018) — индекс как модель функции распределения: «by using neural nets we are able to outperform cache-optimized B-Trees by up to **70%** in speed while saving an order-of-magnitude in memory». Продолжение — PGM-index.
- **van Emde Boas** — $O(\log \log u)$ по размеру вселенной; Pătraşcu и Thorup (STOC 2006) доказали, что в своём режиме это оптимально.

Итог дня: две оси



Весь день мы двигались по **вертикальной** оси, не трогая горизонтальную — пока в 2.8 не сошли с неё совсем.

Точки показаны *порядком*, а не измеренными значениями: в замере №5 при 2^{24} промахов у массива *больше*, чем у дерева (80,6 против 22,4), а времени он тратит втрое меньше.

Что читать (1): доклад и книги

Стержень дня — доклад. Всё, что мы мерили во второй паре, идёт отсюда. Полный список на три слайда вперёд, с датой проверки каждой ссылки, — в `sources.md` рядом с колодой; там же `demo/`: код, посчитавший все числа лекции.

Сергей Слотин, «Optimizing Binary Search», CppCon 2022 youtube.com/watch?v=1RIPMQRBWk

Он же, книга «Algorithms for Modern Hardware» en.algorithmica.org/hpc/data-structures/binary-search/
en.algorithmica.org/hpc/data-structures/s-tree/

Код ко всему докладу github.com/sslotin/amh-code/tree/main/binsearch
Paul Khuong, два поста 2012 г. — pvk.ca/Blog/2012/07/

то, на что ссылается Слотин
Matt Kulukundis, SwissTable, youtube.com/watch?v=ncHmEUmJZf4
CppCon 2017

Книги. Кормен и др. (CLRS), гл. 12 (БДП), гл. 13 (красно-чёрные) и задача 13-3(a) про AVL, гл. 18 (B-деревья); Кнут, т. 3, § 6.2.1–6.2.4 — поиск, деревья, сбалансированные деревья, сильноветвящиеся деревья; Bentley, *Programming Pearls*, 1986, Column 4 — тот самый разбор, в котором напечатано $(L+U) \div 2$; Седжвик, «Алгоритмы на C++», 2014, гл. 13.

Что читать (2): код, который мы открывали

Всё это — рабочий код, который стоит у вас на машине или лежит в открытом зеркале. Номера строк двигаются; ищите по имени функции.

libstdc++	bits/stl_algobase.h — <code>__lower_bound</code> ; bits/stl_tree.h; src/c++98/tree.cc — повороты
libc++	<code>__algorithm/lower_bound.h</code>
glibc	bits/stdlib-bsearch.h — <code>bsearch</code>
CPython	Modules/_bisectmodule.c — <code>bisect</code>
Go	go.dev/src/sort/search.go — комментарий «avoid overflow when computing h»
Rust	library/core/src/slice/mod.rs — <code>binary_search_by</code> и комментарии, объясняющие размен
OpenJDK	java/util/Arrays.java — <code>binarySearch0</code> ; java/util/HashMap.java — <code>TREEIFY_THRESHOLD</code>
Ядро Linux	lib/rbtree.c — пять свойств и набросок доказательства прямо в шапке файла; удобно читать через elixir.bootlin.com/linux/latest/source/
Abseil	absl/container/internal/raw_hash_set.h; записка abseil.io/about/design/swisstables

Зеркала на GitHub: [gcc-mirror/gcc](https://github.com/gcc-mirror/gcc), [torvalds/linux](https://github.com/torvalds/linux), [python/cpython](https://github.com/python/cpython), [rust-lang/rust](https://github.com/rust-lang/rust),
[openjdk/jdk](https://github.com/openjdk/jdk), [abseil/abseil-cpp](https://github.com/abseil/abseil-cpp).

Что читать (3): статьи и первоисточники

Bloch, «Nearly All Binary Searches and Mergesorts are Broken», 2006	research.google/blog/
Bayer, McCreight, <i>Acta Informatica</i> 1 (1972), 173-189 — статья, в которой появились B-деревья	doi.org/10.1007/BF00288683
Khuong, Morin, «Array Layouts for Comparison-Based Searching», JEA 22 (2017) — Эйтцингер	arxiv.org/abs/1509.05053
Kraska et al., «The Case for Learned Index Structures», SIGMOD 2018	arxiv.org/abs/1712.01208
Pătraşcu, Thorup, «Time-Space Trade-Offs for Predecessor Search», STOC 2006 — нижняя граница	arxiv.org/abs/cs/0603043
Leis, Kemper, Neumann, «The Adaptive Radix Tree», ICDE 2013	db.in.tum.de/~leis/papers/ART.pdf
Carter, Wegman, «Universal Classes of Hash Functions», JCSS 18 (1979) — универсальное хеширование	разбор — CLRS §11.3.3
Sedgewick, «Left-Leaning Red-Black Trees», 2008	sedgewick.io/papers/2008LLRB.pdf
RocksDB, «Interpolation search for SST index blocks», 2026	rocksdb.org/blog/

Дальше — слайды, которые читаются, если есть время

Лекция заканчивается предыдущим слайдом. Ниже — сюжеты, которые не поместились: высота случайного дерева, атака на хеш-таблицу, как обе половины дня сошлись в одном классе Java, дерево без указателей и универсальное хеширование.

Высота случайного БДП

- Если n ключей вставлять в случайном порядке, ожидаемая высота дерева — $O(\log n)$ (CLRS, теорема 12.4).
- Не путать с числом сравнений при *поиске*: там у Кнута $\approx 1,386 \lg N$ (мы это видели в 2.1). Высота — другая величина и константа у неё другая, заметно больше.
- При *плохом* порядке высота равна $n - 1$, и «плохой» — это не экзотика, а отсортированный вход.

Вывод для инженера, а не для экзамена: раз вход сортируют чаще, чем перемешивают, «в среднем хорошо» — не аргумент. Балансировку ставят не потому, что среднее плохое, а потому что **худшее** встречается регулярно.

Hash-flooding — когда хеш-таблица становится списком

- Klink и Wälde, 28C3 (2011): если хеш-функция известна, ключи с одинаковым хешем *подбираются*. Все n ключей попадают в одну цепочку — и вставка n элементов стоит $O(n^2)$.
- Это не теория: так клали веб-приложения на PHP, Java, Python, Ruby, отправив один POST-запрос с подобранными именами полей.
- Ответ — **хеш со секретным ключом**: в Rust std по умолчанию SipHash-1-3, ключ случайный на процесс.
- А в самом hashbrown по умолчанию foldhash, который *«does not provide the same level of HashDoS resistance as SipHash»* — и это написано в его же README, прямым текстом.

Размен назван вслух самим автором библиотеки: скорость против устойчивости к подобранным входам. Выбирать — вам.

Java 8 — обе половины дня в одном классе

HashMap в Java 8 (JEP 180) делает вот что:

TREEIFY_THRESHOLD	8	элементов в одной корзине —
MIN_TREEIFY_CAPACITY	64	при таком размере таблицы
⇒ корзина превращается в красно-чёрное дерево		
UNTREEIFY_THRESHOLD	6	— и разворачивается обратно

Хеш-таблица, которая при перегрузке корзины сама достраивает красно-чёрное дерево — и тем закрывает дыру из предыдущего слайда: подобранные коллизии дают $O(\log n)$ вместо $O(n)$.

Обе половины этого дня — в одном классе стандартной библиотеки.

OpenJDK, java/util/HashMap.java, строки 260, 275, 267 (ветка master, прочитано 16.09.2026) — числа E22; замысел — JEP

То же дерево — в десяти строках и без указателей

- Okasaki, «Red-Black Trees in a Functional Setting», JFP 9(4) 1999: балансировка — **четыре образца** и десять строк. Ни родителя, ни цвета в бите указателя: все четыре случая вставки — одно сопоставление с образцом.
- Germane и Might, «**Deletion: The Curse of the Red-Black Tree**», JFP 2014 — одно название уже слайд: удаление и там осталось трудным.
- Красно-чёрное дерево, **проверенное в Coq**: Software Foundations, том 3, глава «Redblack» — инварианты доказаны машиной, а не проверены тестами.

Прямое продолжение сегмента «Откуда мы знаем» из пары 1: там мы доказывали одну строку про переполнение, здесь — целую структуру.

Универсальное хеширование — и сколько стоит словарь

- Картер и Вегман, 1979: функция выбирается **случайно** из семейства, где для любых двух разных ключей вероятность коллизии не больше $1/m$. Противник не может подобрать вход: он не знает, какую функцию мы выберем.
- Теорема: при таком выборе ожидаемая длина цепочки — $1 + \alpha$, где $\alpha = n/m$ — коэффициент заполнения. *Ожидание по выбору функции*, а не по входу: это и есть ответ на предыдущий слайд про hash-flooding.
- Минимальные совершенные хеш-функции: статический словарь без коллизий вовсе. Информационная нижняя граница — $\lg e \approx 1,44$ бита на ключ, и практические конструкции подходят к ней вплотную.